

(12) **United States Patent**
Chujoh et al.

(10) **Patent No.:** **US 12,418,674 B2**
(45) **Date of Patent:** **Sep. 16, 2025**

(54) **VIDEO CODING APPARATUS AND VIDEO DECODING APPARATUS**

(71) Applicant: **Sharp Kabushiki Kaisha, Sakai (JP)**
(72) Inventors: **Takeshi Chujoh, Sakai (JP); Tomohiro Ikai, Sakai (JP); Tomoko Aono, Sakai (JP); Eiichi Sasaki, Sakai (JP); Tomonori Hashimoto, Sakai (JP)**
(73) Assignee: **SHARP KABUSHIKI KAISHA, Sakai (JP)**

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/677,982**

(22) Filed: **May 30, 2024**

(65) **Prior Publication Data**
US 2024/0314347 A1 Sep. 19, 2024

Related U.S. Application Data

(63) Continuation of application No. 17/780,085, filed as application No. PCT/JP2020/043940 on Nov. 26, 2020, now Pat. No. 12,041,263.

(30) **Foreign Application Priority Data**

Dec. 3, 2019 (JP) 2019-218449

(51) **Int. Cl.**
H04N 19/513 (2014.01)

(52) **U.S. Cl.**
CPC **H04N 19/513** (2014.11)

(58) **Field of Classification Search**
None
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

8,331,440 B2 *	12/2012	Song		H04N 19/85
				375/240.03
2013/0101026 A1 *	4/2013	Song		H04N 19/61
				375/240.03
2019/0313094 A1 *	10/2019	Kanoh		H04N 19/159
2019/0387251 A1 *	12/2019	Lin		H04N 19/105
2020/0036973 A1 *	1/2020	Kanoh		H04N 19/176
2020/0036974 A1 *	1/2020	Kanoh		H04N 19/182
2020/0036976 A1 *	1/2020	Kanoh		H04N 19/157
2020/0053382 A1 *	2/2020	Abe		H04N 19/105
2020/0099952 A1 *	3/2020	Kanoh		H04N 19/543

OTHER PUBLICATIONS

Chujoh et al., "Video Coding Apparatus and Video Decoding Apparatus", U.S. Appl. No. 17/780,085, filed May 26, 2022.

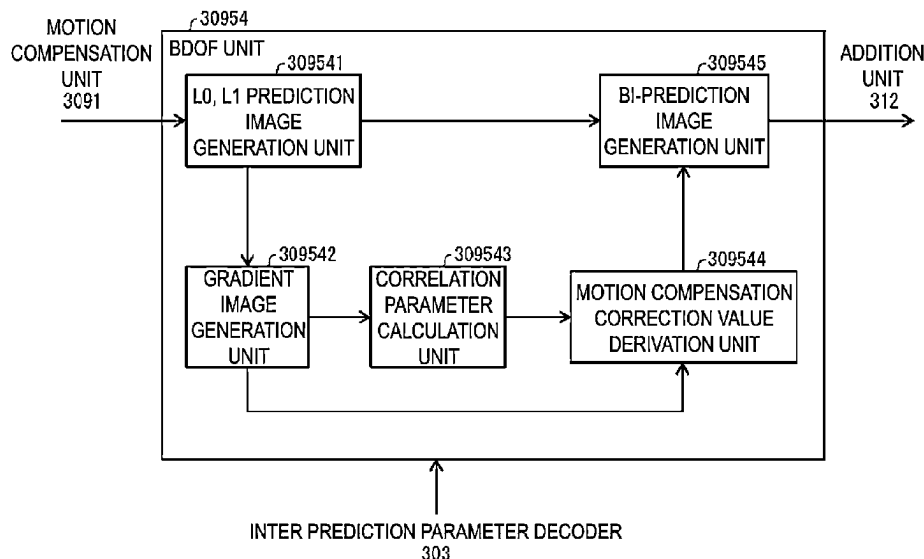
* cited by examiner

Primary Examiner — Mohammad J Rahman
(74) *Attorney, Agent, or Firm* — Keating & Bennett, LLP

(57) **ABSTRACT**

In the related art, in some cases the weighted prediction processing is performed even in a case that normal prediction processing is to be performed. A video decoding apparatus includes a weighted prediction processing unit configured to decode a weight coefficient and an offset value from coded data and to generate a prediction image by multiplying an interpolation image by the weight coefficient and adding the offset value to the interpolation image, and a normal prediction processing unit configured to generate a prediction image from the interpolation image. In a case that in the weighted prediction processing unit, a bi-prediction is performed and information of the weight coefficient and the offset value is absent for both a reference list 0 and a reference list 1, the normal prediction processing unit generates a bi-prediction image.

1 Claim, 30 Drawing Sheets



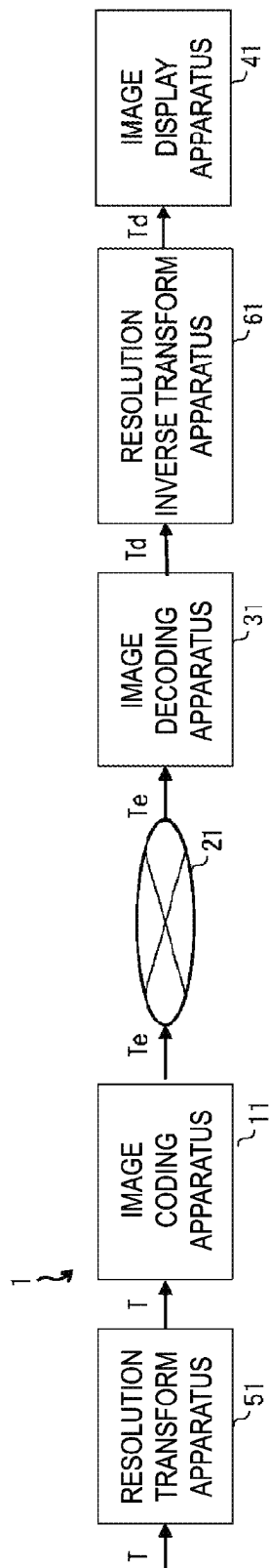


FIG. 1

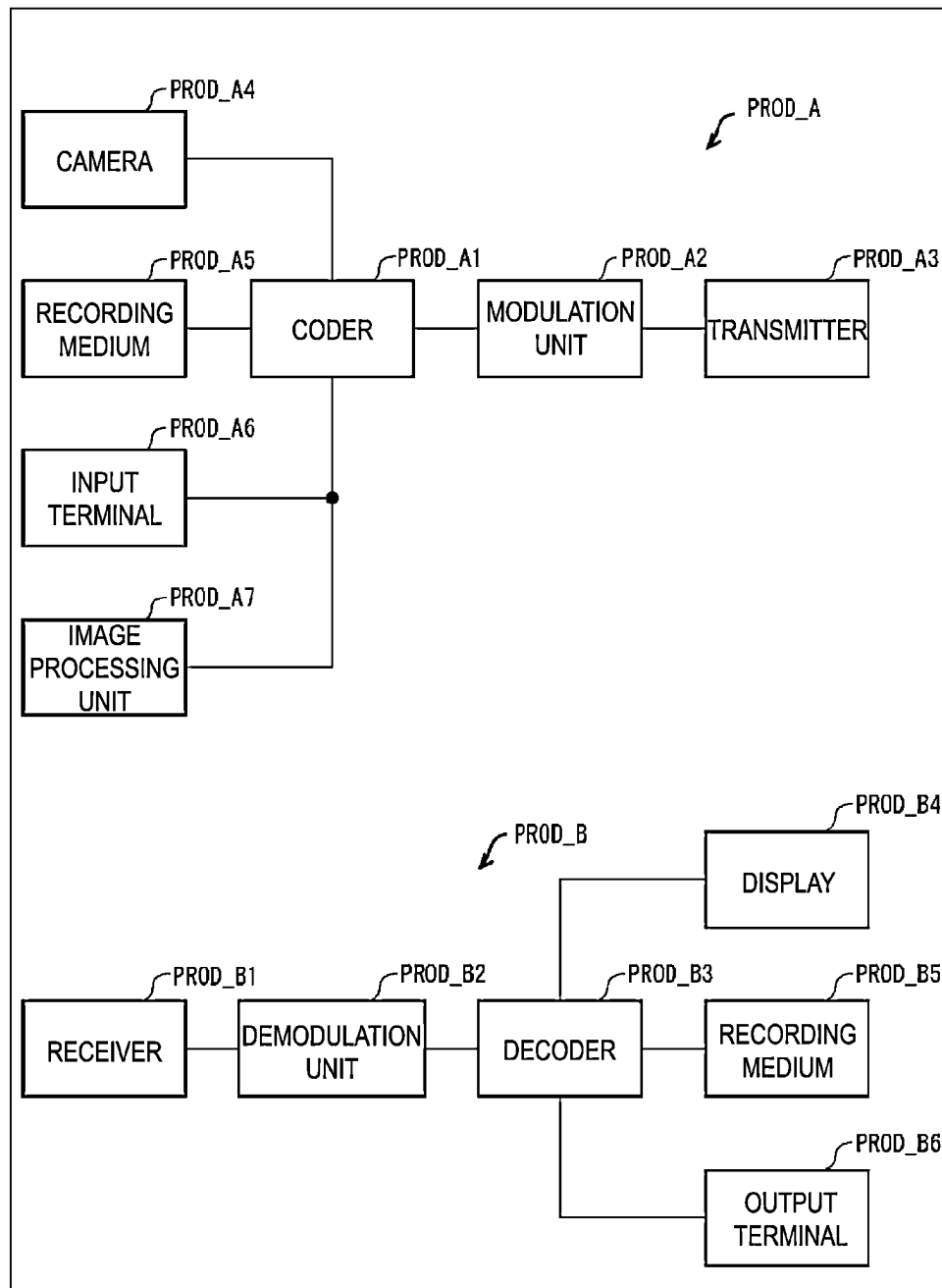


FIG. 2

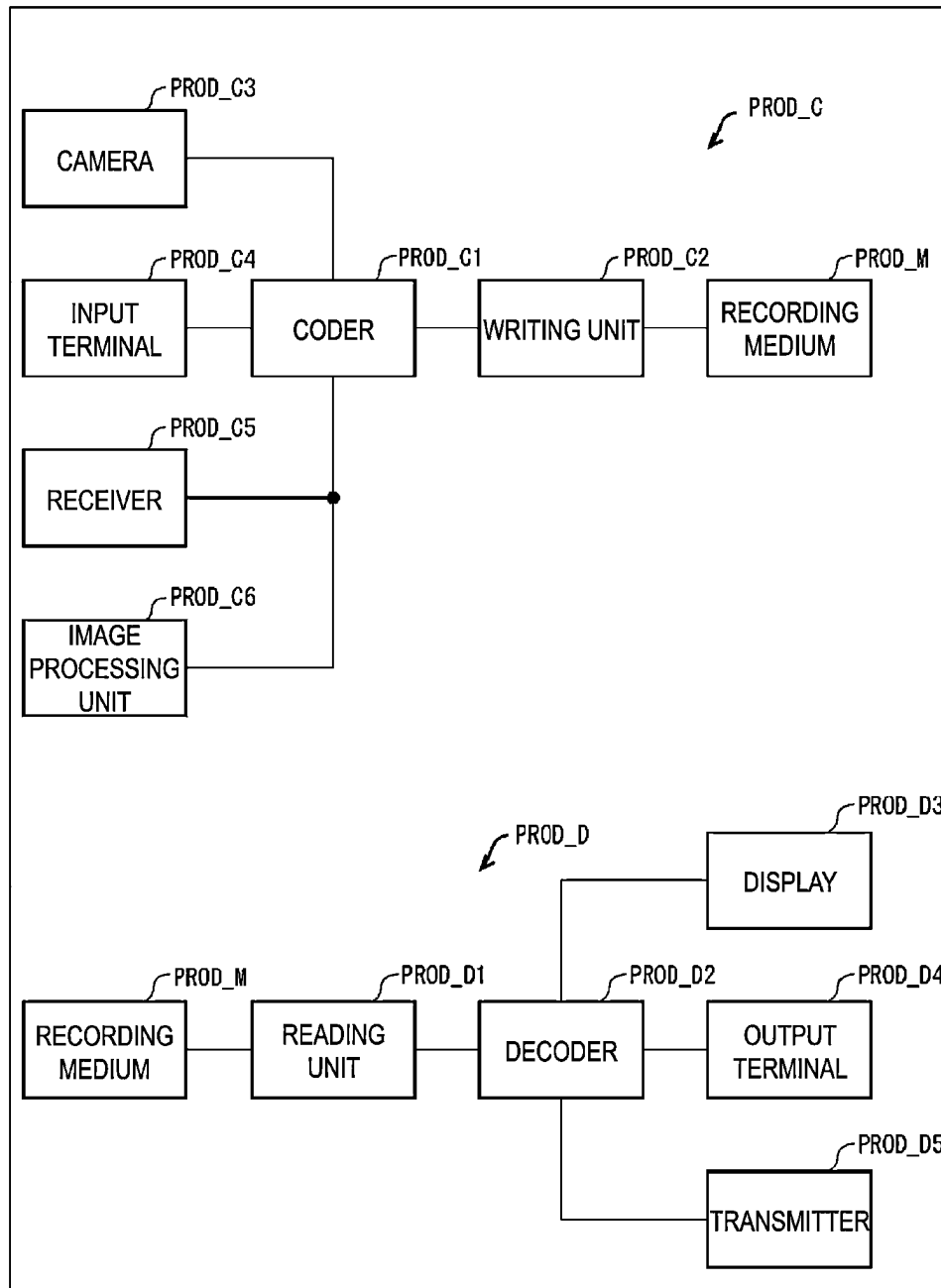


FIG. 3

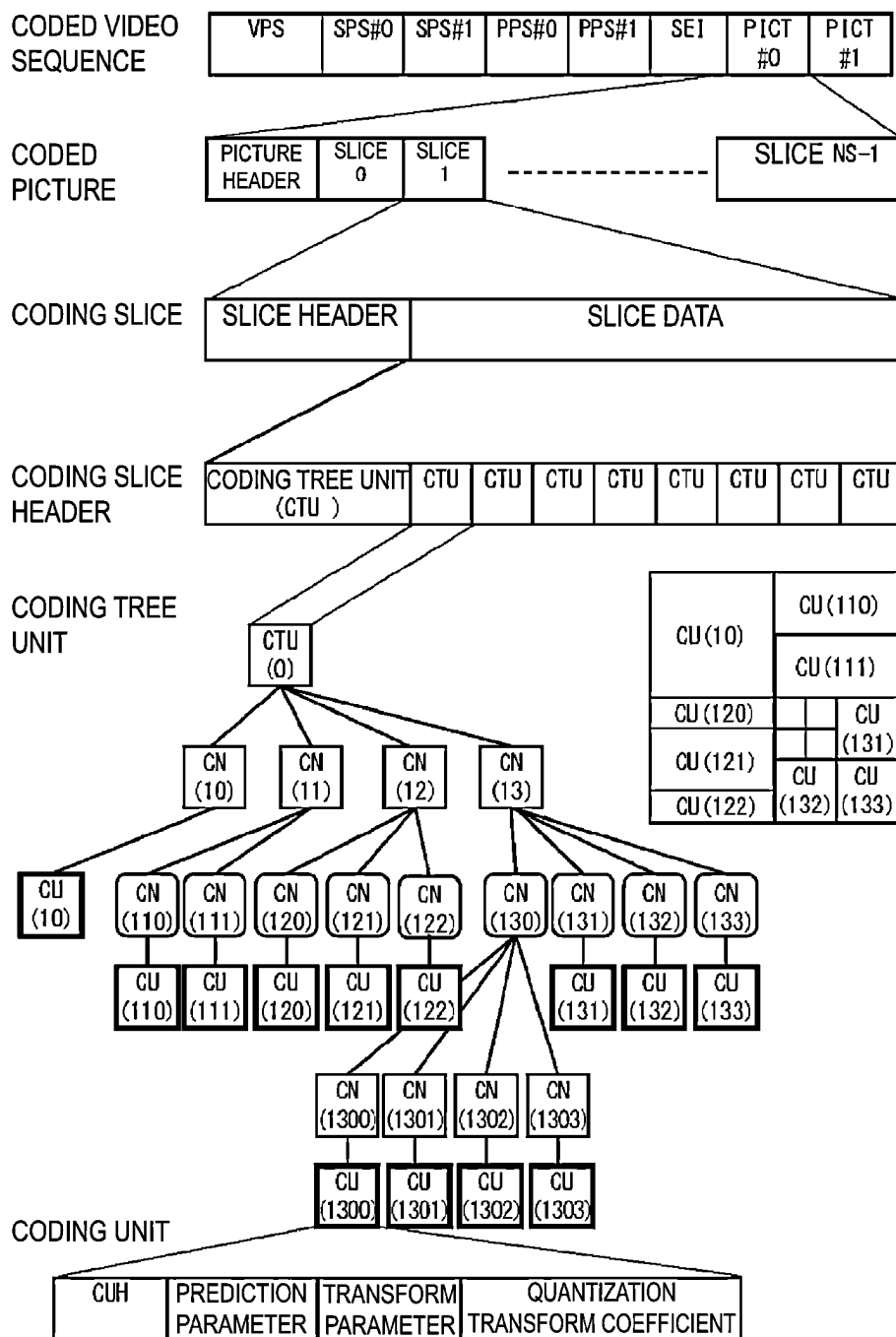
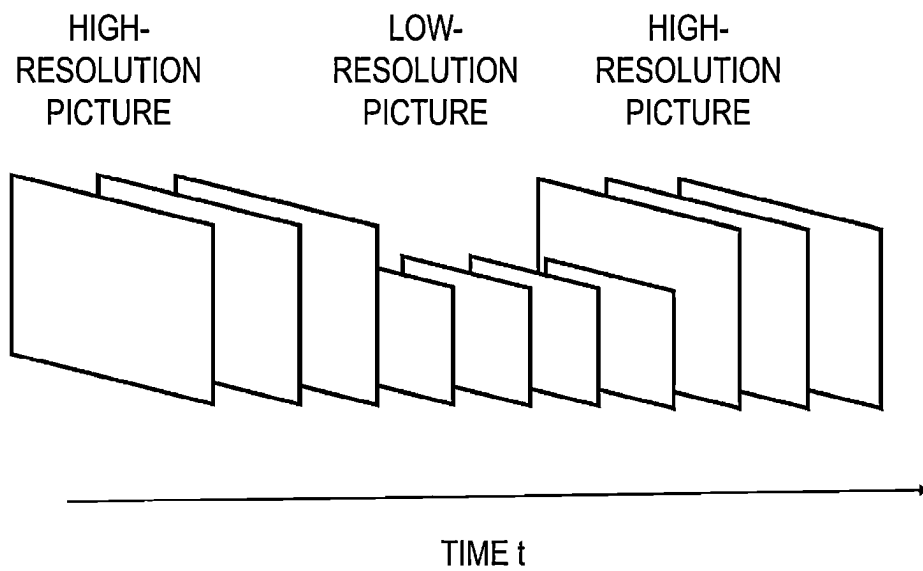
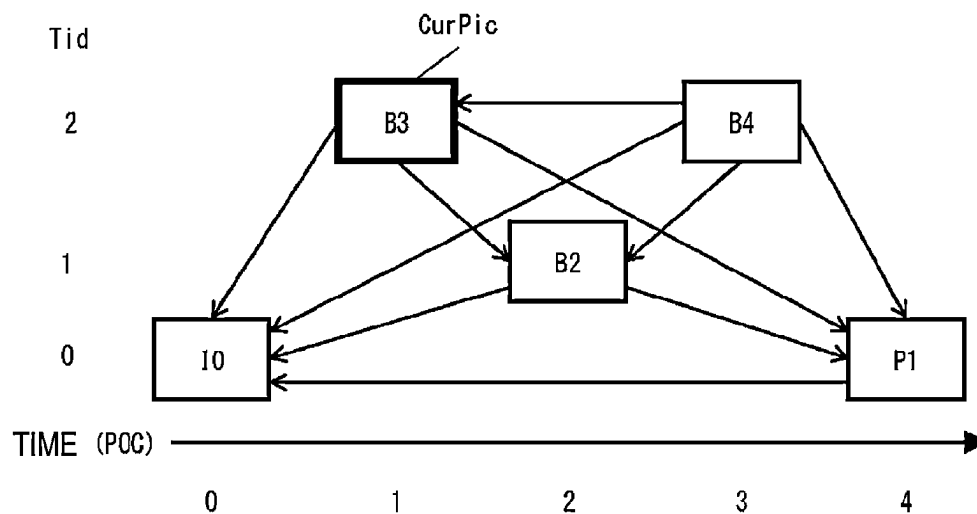


FIG. 4

**FIG. 5**



REFERENCE PICTURE LIST OF CurPic (= B3)

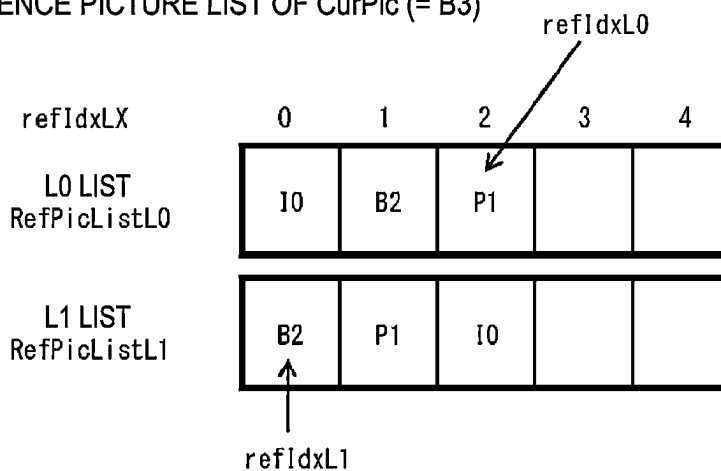


FIG. 6

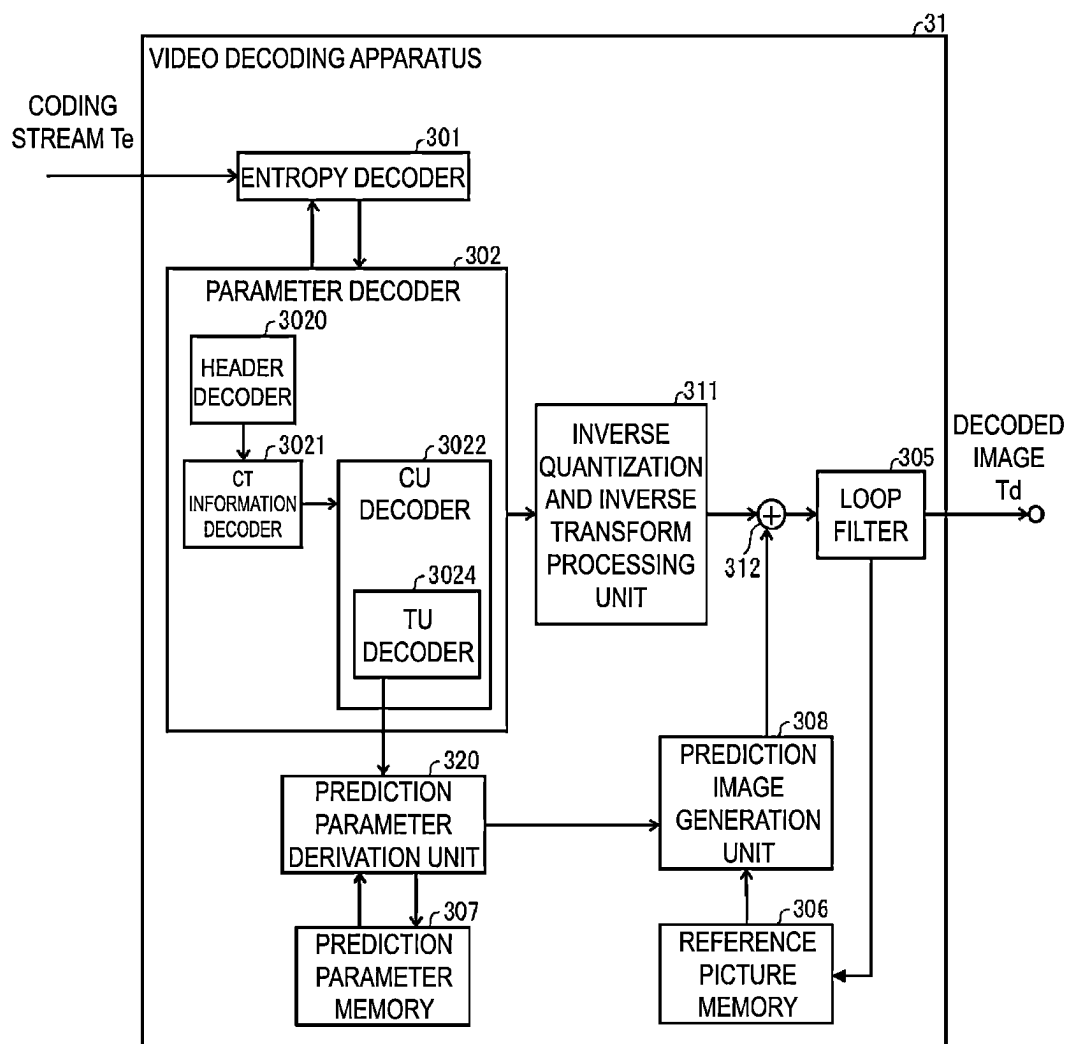


FIG. 7

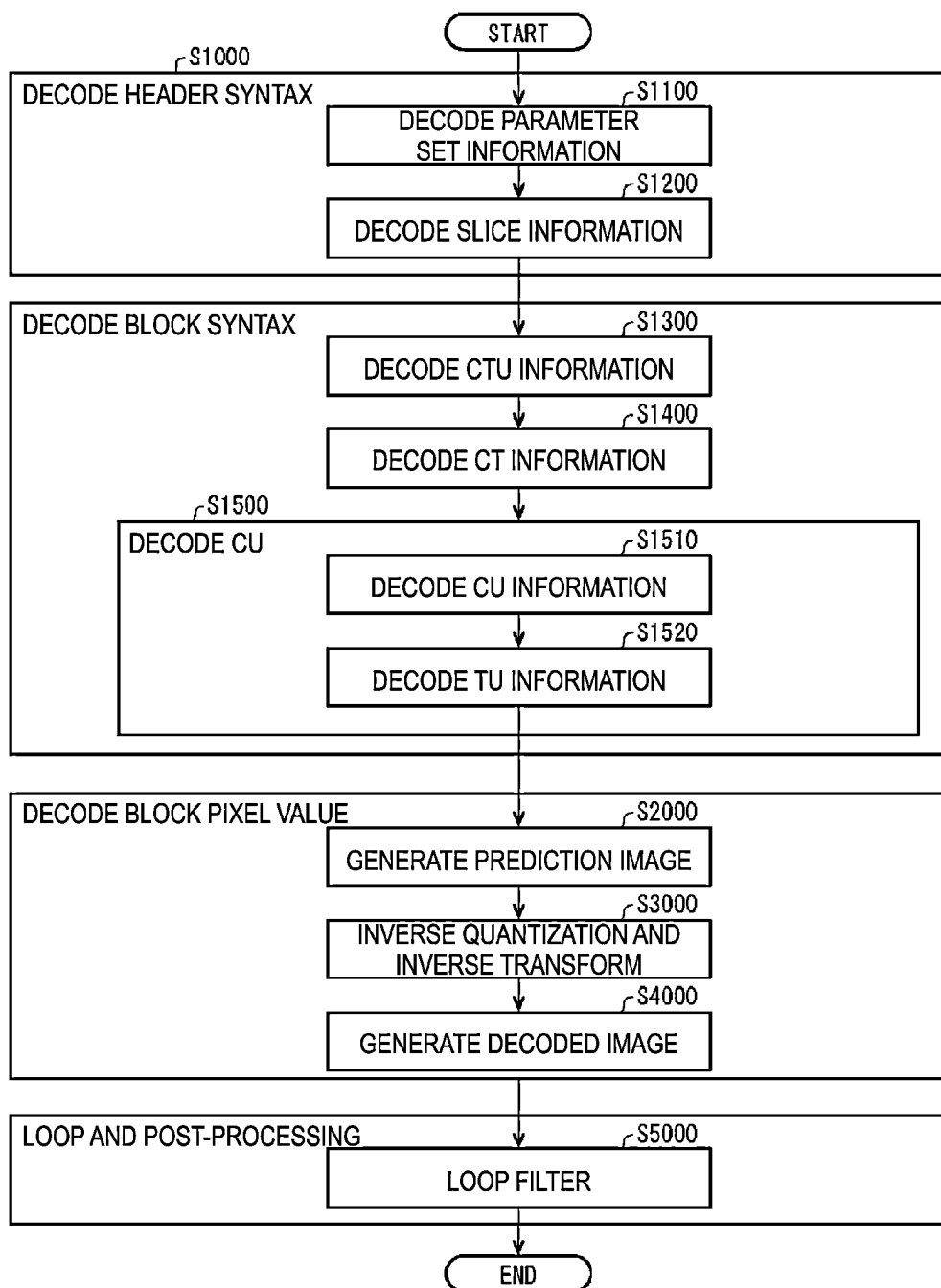


FIG. 8

slice_header() {	Descriptor
...	
if(pic_temporal_mvp_enabled_flag) {	
if(slice_type == B && !pps_collocated_from_l0_idc)	
collocated_from_l0_flag	u(1)
if((collocated_from_l0_flag && NumRefIdxActive[0] > 1)	
(!collocated_from_l0_flag && NumRefIdxActive[1] > 1))	
collocated_ref_idx	ue(v)
}	
...	
}	

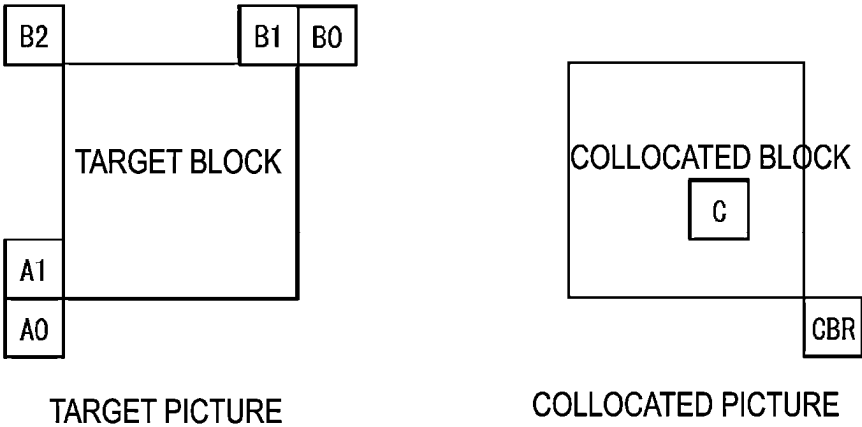


FIG. 9

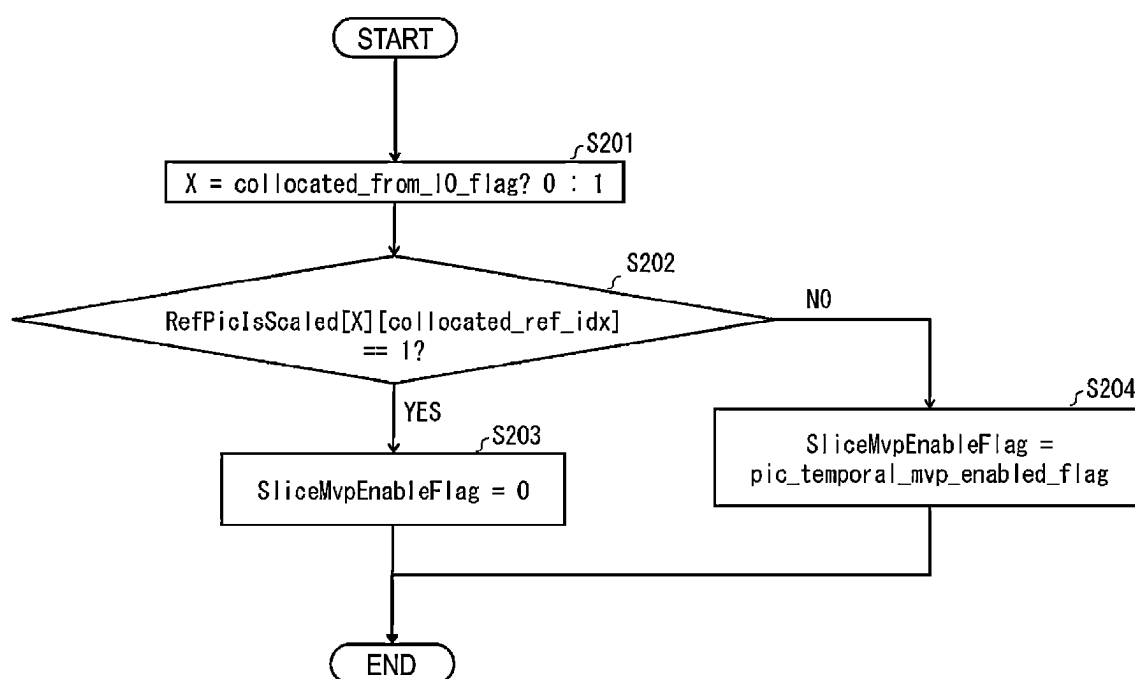


FIG. 10

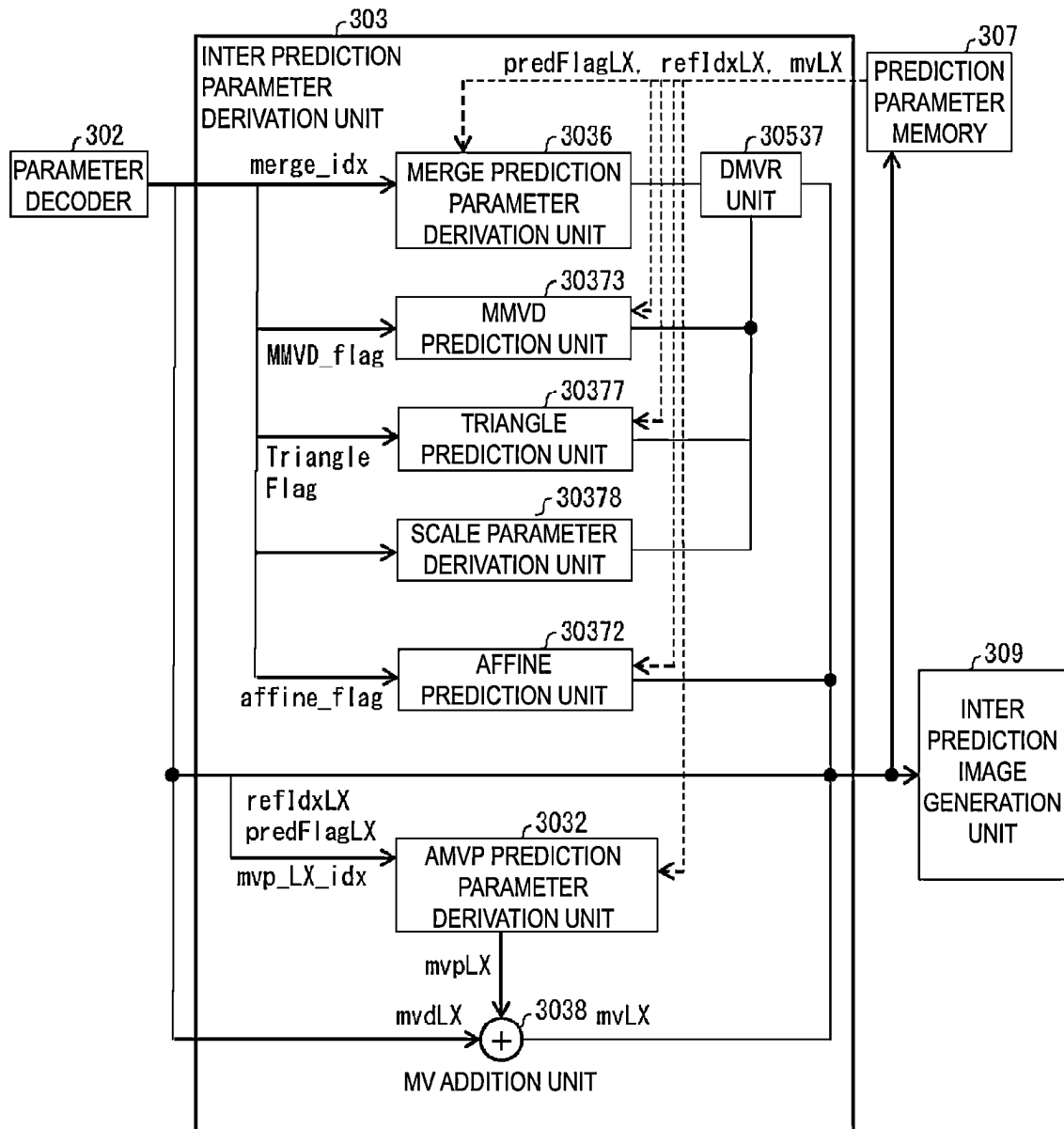


FIG. 11

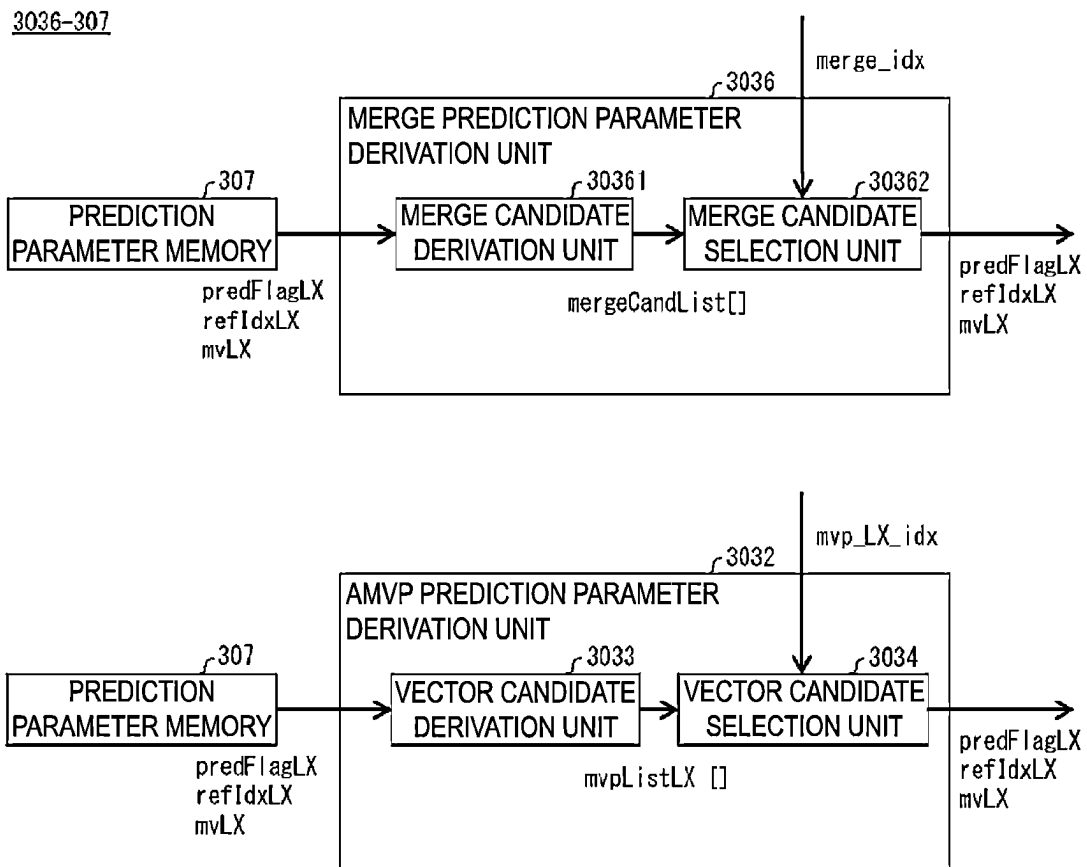


FIG. 12

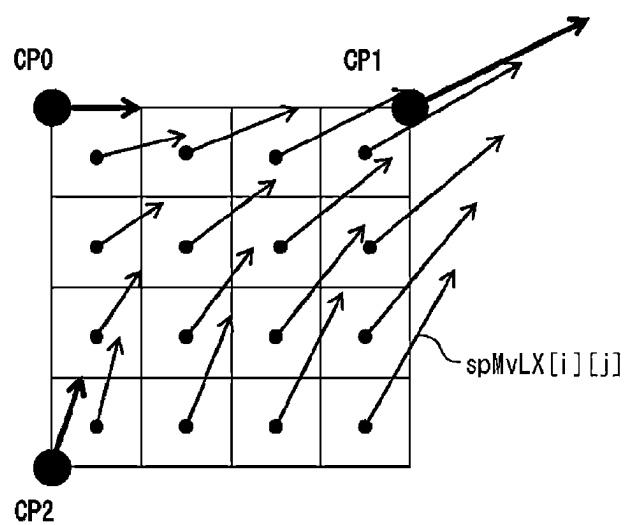


FIG. 13

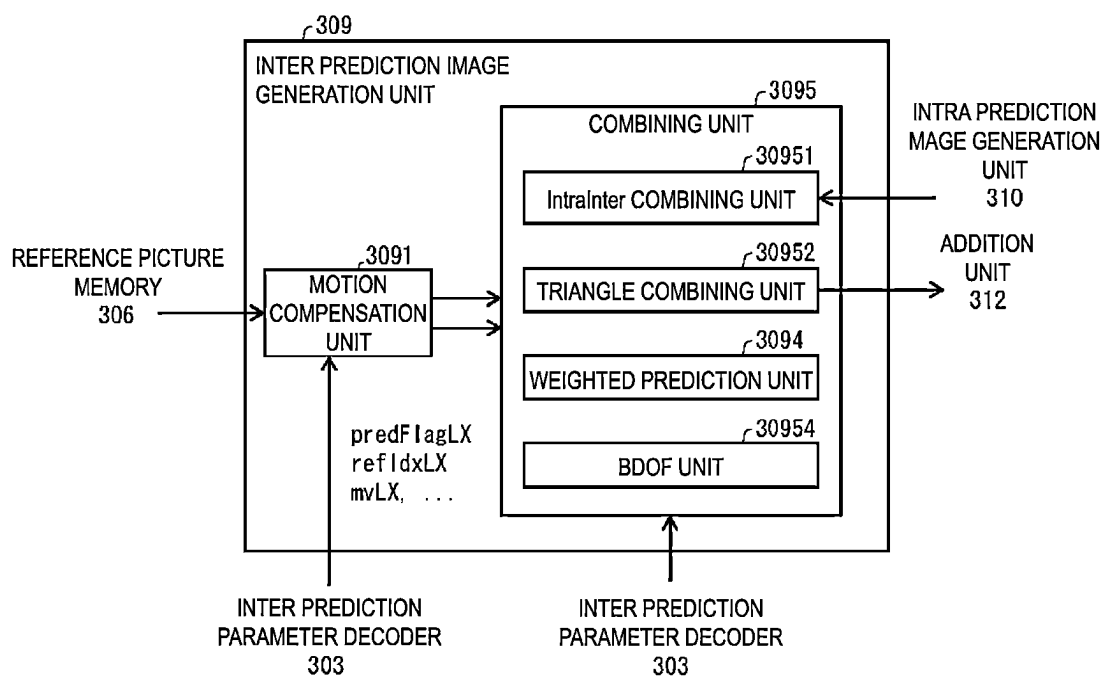


FIG. 14

Fractional sample position p	interpolation filter coefficients							
	$f_r[p][0]$	$f_r[p][1]$	$f_r[p][2]$	$f_r[p][3]$	$f_r[p][4]$	$f_r[p][5]$	$f_r[p][6]$	$f_r[p][7]$
1	0	1	-3	63	4	-2	1	0
2	-1	2	-5	62	8	-3	1	0
3	-1	3	-8	60	13	-4	1	0
4	-1	4	-10	58	17	-5	1	0
5	-1	4	-11	52	26	-8	3	-1
6	-1	3	-9	47	31	-10	4	-1
7	-1	4	-11	45	34	-10	4	-1
8 (hpellfdx == 0)	-1	4	-11	40	40	-11	4	-1
8 (hpellfdx == 1)	0	3	9	20	20	9	3	0
9	-1	4	-10	34	45	-11	4	-1
10	-1	4	-10	31	47	-9	3	-1
11	-1	3	-8	26	52	-11	4	-1
12	0	1	-5	17	58	-10	4	-1
13	0	1	-4	13	60	-8	3	-1
14	0	1	-3	8	62	-5	2	-1
15	0	1	-2	4	63	-3	1	0

FIG. 15

Fractional sample position p	interpolation filter coefficients							
	$f_r[p][0]$	$f_r[p][1]$	$f_r[p][2]$	$f_r[p][3]$	$f_r[p][4]$	$f_r[p][5]$	$f_r[p][6]$	$f_r[p][7]$
1	0	1	-3	63	4	-2	1	0
2	0	1	-5	62	8	-3	1	0
3	0	2	-8	60	13	-4	1	0
4	0	3	-10	58	17	-5	1	0
5	0	3	-11	52	26	-8	2	0
6	0	2	-9	47	31	-10	3	0
7	0	3	-11	45	34	-10	3	0
8	0	3	-11	40	40	-11	3	0
9	0	3	-10	34	45	-11	3	0
10	0	3	-10	31	47	-9	2	0
11	0	2	-8	26	52	-11	3	0
12	0	1	-5	17	58	-10	3	0
13	0	1	-4	13	60	-8	2	0
14	0	1	-3	8	62	-5	1	0
15	0	1	-2	4	63	-3	1	0

FIG. 16

Fractional sample position p	interpolation filter coefficients							
	$f_r[p][0]$	$f_r[p][1]$	$f_r[p][2]$	$f_r[p][3]$	$f_r[p][4]$	$f_r[p][5]$	$f_r[p][6]$	$f_r[p][7]$
0	-1	-5	17	42	17	-5	-1	0
1	0	-5	15	41	19	-5	-1	0
2	0	-5	13	40	21	-4	-1	0
3	0	-5	11	39	24	-4	-2	1
4	0	-5	9	38	26	-3	-2	1
5	0	-5	7	38	28	-2	-3	1
6	1	-5	5	36	30	-1	-3	1
7	1	-4	3	35	32	0	-4	1
8	1	-4	2	33	33	2	-4	1
9	1	-4	0	32	35	3	-4	1
10	1	-3	-1	30	36	5	-5	1
11	1	-3	-2	28	38	7	-5	0
12	1	-2	-3	26	38	9	-5	0
13	1	-2	-4	24	39	11	-5	0
14	0	-1	-4	21	40	13	-5	0
15	0	-1	-5	19	41	15	-5	0

FIG. 17

Fractional sample position p	interpolation filter coefficients							
	$f_r[p][0]$	$f_r[p][1]$	$f_r[p][2]$	$f_r[p][3]$	$f_r[p][4]$	$f_r[p][5]$	$f_r[p][6]$	$f_r[p][7]$
0	-4	2	20	28	20	2	-4	0
1	-4	0	19	29	21	5	-4	-2
2	-4	-1	18	29	22	6	-4	-2
3	-4	-1	16	29	23	7	-4	-2
4	-4	-1	16	28	24	7	-4	-2
5	-4	-1	14	28	25	8	-4	-2
6	-3	-3	14	27	26	9	-3	-3
7	-3	-1	12	28	25	10	-4	-3
8	-3	-3	11	27	27	11	-3	-3
9	-3	-4	10	25	28	12	-1	-3
10	-3	-3	9	26	27	14	-3	-3
11	-2	-4	8	25	28	14	-1	-4
12	-2	-4	7	24	28	16	-1	-4
13	-2	-4	7	23	29	16	-1	-4
14	-2	-4	6	22	29	18	-1	-4
15	-2	-4	5	21	29	19	0	-4

FIG. 18

Fractional sample position p	interpolation filter coefficients			
	$f_c[p][0]$	$f_c[p][1]$	$f_c[p][2]$	$f_c[p][3]$
1	-1	63	2	0
2	-2	62	4	0
3	-2	60	7	-1
4	-2	58	10	-2
5	-3	57	12	-2
6	-4	56	14	-2
7	-4	55	15	-2
8	-4	54	16	-2
9	-5	53	18	-2
10	-6	52	20	-2
11	-6	49	24	-3
12	-6	46	28	-4
13	-5	44	29	-4
14	-4	42	30	-4
15	-4	39	33	-4
16	-4	36	36	-4
17	-4	33	39	-4
18	-4	30	42	-4
19	-4	29	44	-5
20	-4	28	46	-6
21	-3	24	49	-6
22	-2	20	52	-6
23	-2	18	53	-5
24	-2	16	54	-4
25	2	15	55	4
26	-2	14	56	-4
27	-2	12	57	-3
28	-2	10	58	-2
29	-1	7	60	-2
30	0	4	62	-2
31	0	2	63	-1

FIG. 19

Fractional sample position p	interpolation filter coefficients			
	$f_c[p][0]$	$f_c[p][1]$	$f_c[p][2]$	$f_c[p][3]$
0	12	40	12	0
1	11	40	13	0
2	10	40	15	-1
3	9	40	16	-1
4	8	40	17	-1
5	8	39	18	-1
6	7	39	19	-1
7	6	38	21	-1
8	5	38	22	-1
9	4	38	23	-1
10	4	37	24	-1
11	3	36	25	0
12	3	35	26	0
13	2	34	28	0
14	2	33	29	0
15	1	33	30	0
16	1	31	31	1
17	0	30	33	1
18	0	29	33	2
19	0	28	34	2
20	0	26	35	3
21	0	25	36	3
22	-1	24	37	4
23	-1	23	38	4
24	-1	22	38	5
25	-1	21	38	6
26	-1	19	39	7
27	-1	18	39	8
28	-1	17	40	8
29	-1	16	40	9
30	-1	15	40	10
31	0	13	40	11

FIG. 20

Fractional sample position p	interpolation filter coefficients			
	$f_c[p][0]$	$f_c[p][1]$	$f_c[p][2]$	$f_c[p][3]$
0	17	30	17	0
1	17	30	18	-1
2	16	30	18	0
3	16	30	18	0
4	15	30	18	1
5	14	30	18	2
6	13	29	19	3
7	13	29	19	3
8	12	29	20	3
9	11	28	21	4
10	10	28	22	4
11	10	27	22	5
12	9	27	23	5
13	9	26	24	5
14	8	26	24	6
15	7	26	25	6
16	7	25	25	7
17	6	25	26	7
18	6	24	26	8
19	5	24	26	9
20	5	23	27	9
21	5	22	27	10
22	4	22	28	10
23	4	21	28	11
24	3	20	29	12
25	3	19	29	13
26	3	19	29	13
27	2	18	30	14
28	1	18	30	15
29	0	18	30	16
30	0	18	30	16
31	-1	18	30	17

FIG. 21

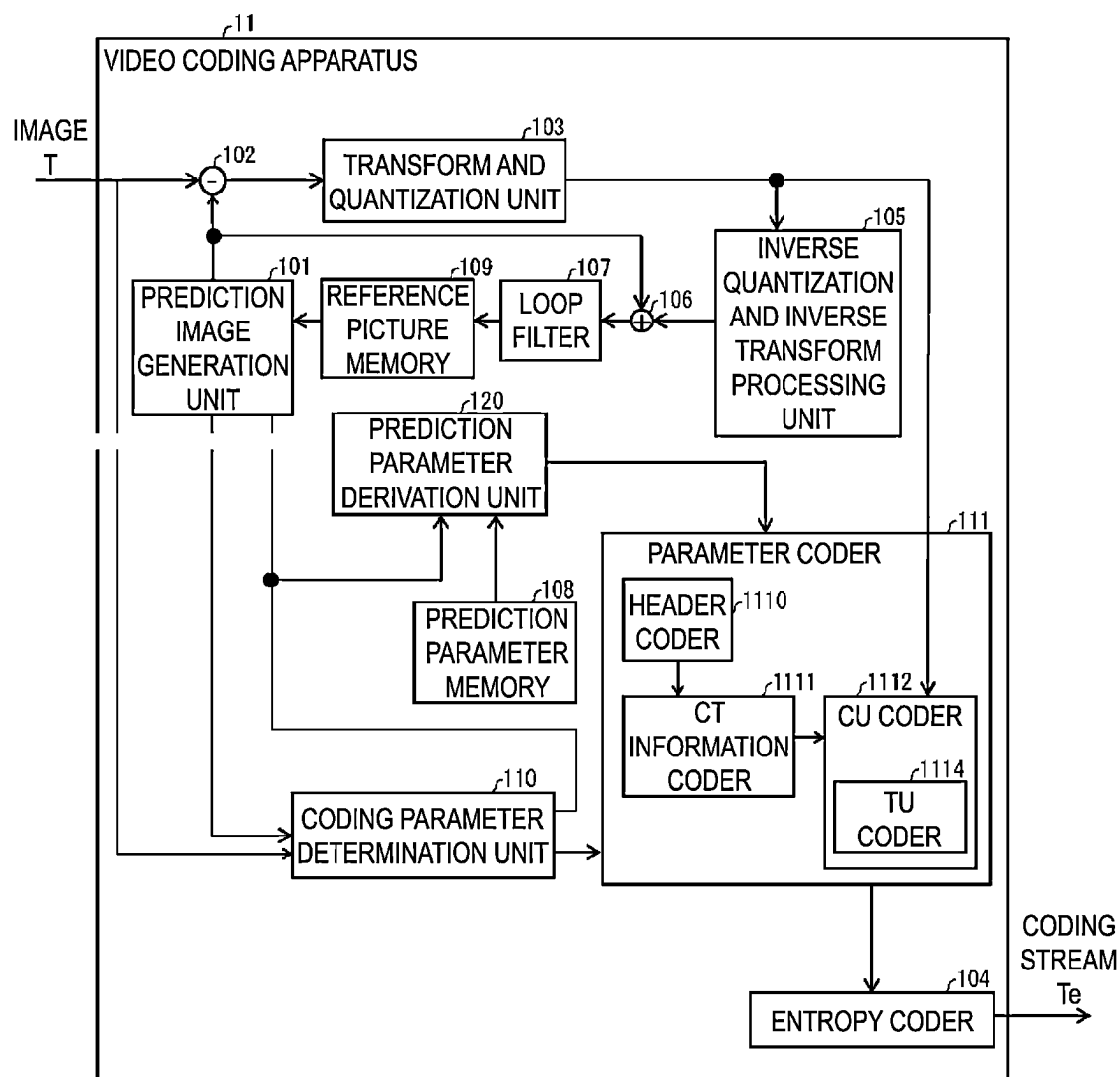


FIG. 22

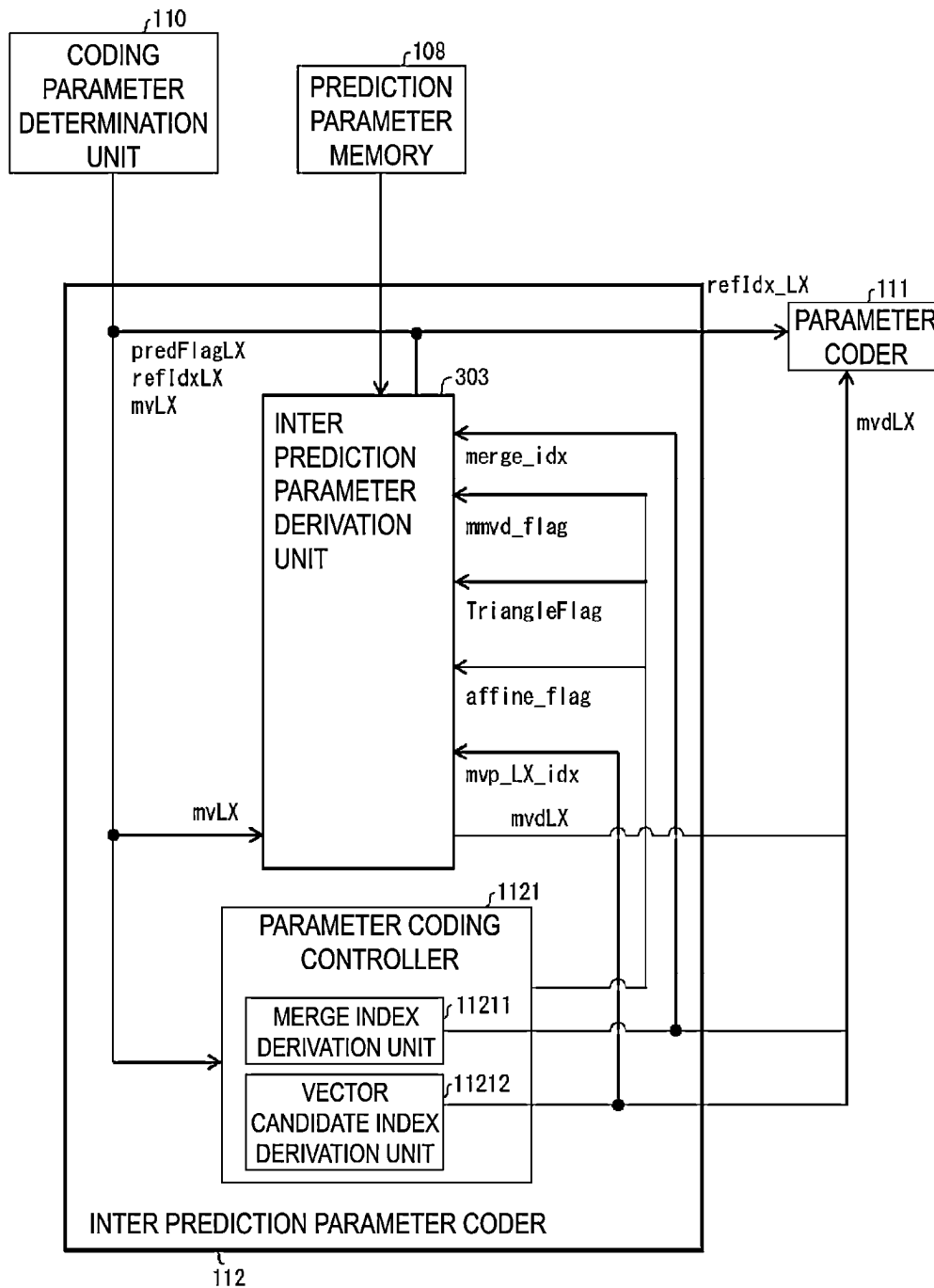


FIG. 23

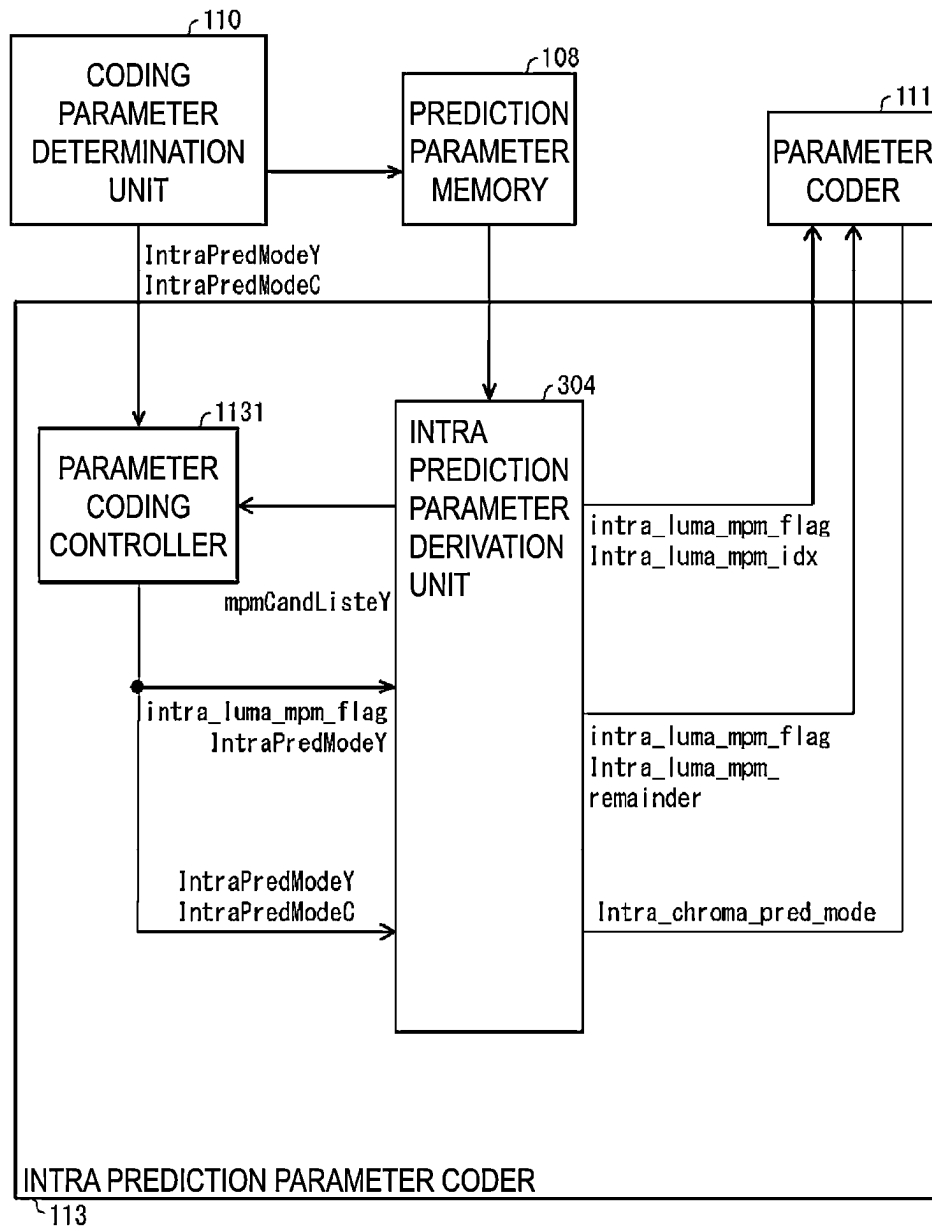


FIG. 24

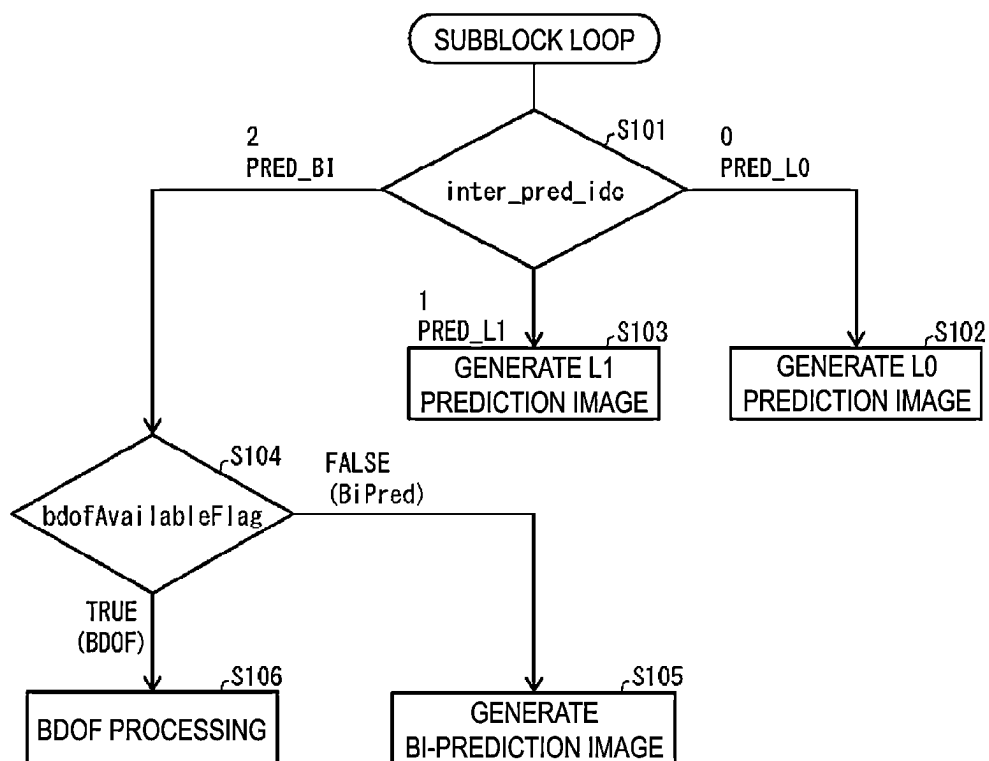


FIG. 25

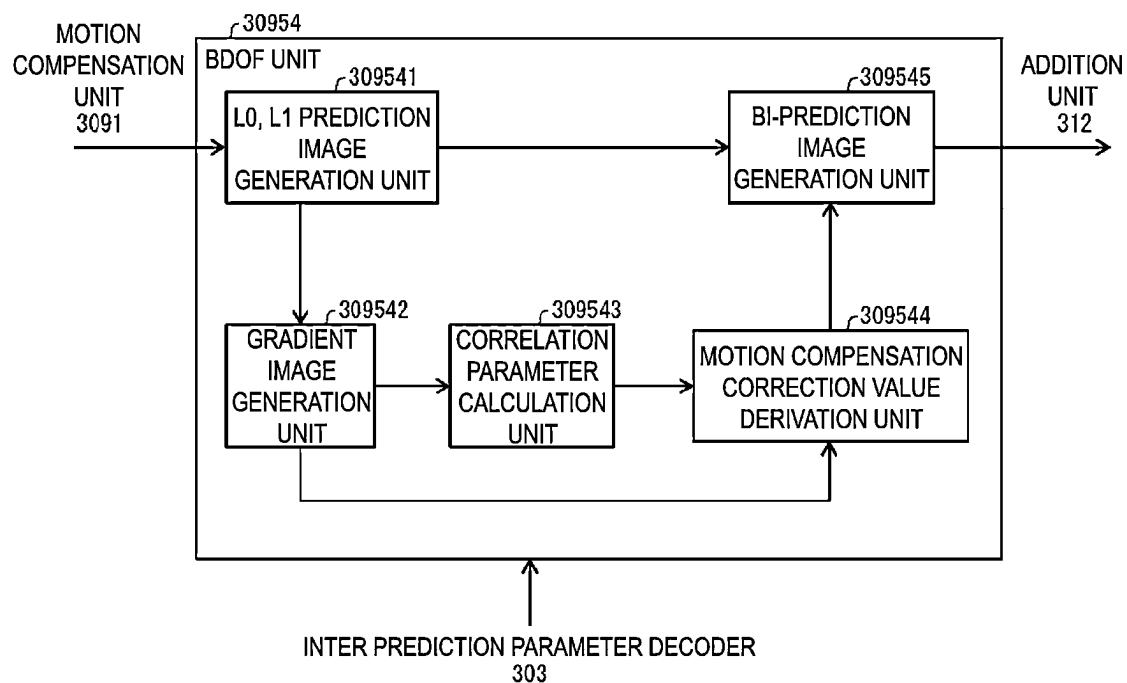


FIG. 26

(a)

seq parameter set rbsp() {	Descriptor
sps decoding parameter set id	ue(4)
sps video parameter set id	ue(4)
...	
sps weighted pred flag	u(1)
sps weighted bipred flag	u(1)
...	
}	

(b)

pic parameter set rbsp() {	Descriptor
pps pic parameter set id	ue(v)
pps seq parameter set id	ue(v)
...	
pps weighted pred flag	u(1)
pps weighted bipred flag	u(1)
...	
}	

(c)

slice header() {	Descriptor
...	
if(slice type != I) {	
...	
if((pps_weighted_pred_flag && slice_type == P) (pps_weighted_bipred_flag && slice_type == B))	
pred_weight_table()	
}	
...	
}	

FIG. 27

pred weight table() {	Descriptor
luma_log2_weight_denom	ue(v)
if(ChromaArrayType != 0)	
delta_chroma_log2_weight_denom	se(v)
for(i = 0; i < NumRefIdxActive[0]; i++)	
luma_weight_l0_flag[i]	u(1)
if(ChromaArrayType != 0)	
for(i = 0; i < NumRefIdxActive[0]; i++)	
chroma_weight_l0_flag[i]	u(1)
for(i = 0; i < NumRefIdxActive[0]; i++) {	
if(luma_weight_l0_flag[i]) {	
delta_luma_weight_l0[i]	se(v)
luma_offset_l0[i]	se(v)
}	
if(chroma_weight_l0_flag[i])	
for(j = 0; j < 2; j++) {	
delta_chroma_weight_l0[i][j]	se(v)
delta_chroma_offset_l0[i][j]	se(v)
}	
}	
if(slice_type == B) {	
for(i = 0; i < NumRefIdxActive[1]; i++)	
luma_weight_l1_flag[i]	u(1)
if(ChromaArrayType != 0)	
for(i = 0; i < NumRefIdxActive[1]; i++)	
chroma_weight_l1_flag[i]	u(1)
for(i = 0; i < NumRefIdxActive[1]; i++) {	
if(luma_weight_l1_flag[i]) {	
delta_luma_weight_l1[i]	se(v)
luma_offset_l1[i]	se(v)
}	
if(chroma_weight_l1_flag[i])	
for(j = 0; j < 2; j++) {	
delta_chroma_weight_l1[i][j]	se(v)
delta_chroma_offset_l1[i][j]	se(v)
}	
}	
}	
}	

FIG. 28

coding_unit(x0, y0, cbWidth, cbHeight, cqtDepth, treeType, modeType) {	Descriptor
...	
if(slice_type != 1 sps_ibc_enabled_flag sps_palette_enabled_flag) {	
...	
if(sps_bcw_enabled_flag && inter_pred_idc[x0][y0] == PRED_BI && !WeightedPredFlag(B, 0, ref_idx_10, ref_idx_11) && !WeightedPredFlag(B, 1, ref_idx_10, ref_idx_11) && cbWidth * cbHeight >= 256)	
bcw_idx[x0][y0]	ae(v)
}	
...	
}	

FIG. 29

(a)

	Descriptor
seq parameter set rbsp() {	
sps decoding parameter set id	ue(4)
sps video parameter set id	ue(4)
...	
sps weighted pred flag	u(1)
sps weighted bipred flag	u(1)
...	
}	

(b)

	Descriptor
picture header rbsp() {	
...	
ph pic parameter set id	ue(v)
...	
if (sps weighted pred flag)	
pic weighted pred flag	u(1)
if (sps weighted bipred flag)	
pic weighted hipred flag	u(1)
...	
}	

(c)

	Descriptor
slice header() {	
...	
if(slice_type != I) {	
...	
if((pic_weighted_pred_flag && slice_type == P) (pic_weighted_bipred_flag && slice_type == B))	
pred_weight_table()	
}	
...	
}	

FIG. 30

VIDEO CODING APPARATUS AND VIDEO
DECODING APPARATUS

TECHNICAL FIELD

The embodiments of the present invention relate to a prediction image generation apparatus, a video decoding apparatus, and a video coding apparatus.

BACKGROUND ART

A video coding apparatus which generates coded data by coding a video, and a video decoding apparatus which generates decoded images by decoding the coded data are used for efficient transmission or recording of videos.

Specific video coding schemes include, for example, H.264/AVC scheme, H.265/High-Efficiency Video Coding (HEVC) scheme, and the like.

In such a video coding scheme, images (pictures) constituting a video are managed in a hierarchical structure including slices obtained by splitting an image, coding tree units (CTUs) obtained by splitting a slice, units of coding (coding units; which will be referred to as CUs) obtained by splitting a coding tree unit, and transform units (TUs) obtained by splitting a coding unit, and are coded/decoded for each CU.

In such a video coding scheme, usually, a prediction image is generated based on a local decoded image that is obtained by coding/decoding an input image (a source image), and prediction errors (which may be referred to also as “difference images” or “residual images”) obtained by subtracting the prediction image from the input image are coded. Generation methods of prediction images include an inter-picture prediction (inter prediction) and an intra-picture prediction (intra prediction).

In addition, the recent technology for video coding and decoding includes NPL 1.

In NPL 1, weighted prediction is switched by the variable weightedPredFlag. In a case that the slice_type is set equal to P, weightedPredFlag is set equal to pps_weighted_pred_flag. Otherwise (in a case that slice_type is equal to B), weightedPredFlag is set equal to (pps_weighted_bipred_flag && !dmvrFlag). Here, pps_weighted_pred_flag is a flag indicating whether the weighted prediction applies in a case that slice_type is P, and is a variable defined in the Picture Parameter Set. pps_weighted_bipred_flag is a flag indicating whether the weighted prediction applies in a case that slice_type is B, and is a variable defined in the Picture Parameter Set. The variable dmvrFlag is a variable indicating whether DMVR processing described below is performed.

The weighted prediction processing is invoked depending on the value of the variable weightedPredFlag. In a case that weightedPredFlag is equal to 0, or bcwIdx is not equal to 0, then normal prediction processing is invoked, and otherwise (in a case that weightedPredFlag is 1 and that bcwIdx is 0), then the processing of weighted prediction is invoked.

CITATION LIST

Non Patent Literature

NPL 1: “Versatile Video Coding (Draft 7),” JVET-P2001-vE, Joint Video Exploration Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 2019-11-14

SUMMARY OF INVENTION

Technical Problem

Even in a case that the value of the variable weightedPredFlag is 1 and hence the weighted prediction processing is invoked, however, there is a case where the normal prediction processing is actually to be performed. The method described in NPL 1 has a problem in that even in the case described above, the weighted prediction processing is performed.

Specifically, in NPL 1, a flag is present that indicates whether a weight coefficient and an offset value for luminance and chrominance are present for each reference picture in an L0 list and an L1 list, whereas only one right shift value (denoted as X) corresponding to a denominator of the weight coefficient is present for each of the luminance and the chrominance. Thus, in a case that no weight coefficient and no offset value for the reference list is present, the weighted prediction processing is performed with the value of the weight coefficient set equal to the Xth power of 2 and with the offset value set equal to 0. Thus, in a case that no weight coefficient and no offset value for the reference list is present in L0 prediction or the L1 prediction, the weighted prediction processing is performed with the value of the weight coefficient set equal to the Xth power of 2 and with the offset value set equal to 0 even though the normal prediction processing is originally to be performed. In a case that no weight coefficient and no offset value is present for both the L0 list and the L1 list in bi-prediction, the weighted prediction processing is performed with the value of the weight coefficient set equal to the Xth power of 2 and with the offset value set equal to 0 even though the normal bi-prediction processing is originally to be performed. As described above, there is a problem in that, in some cases the weighted prediction processing is performed even in a case that the normal prediction processing is to be performed.

Solution to Problem

A video decoding apparatus according to an aspect of the present invention includes:

- a weighted prediction processing unit configured to decode a weight coefficient and an offset value from coded data and to generate a prediction image by multiplying an interpolation image by the weight coefficient and adding the offset value to the interpolation image, and
 - a normal prediction processing unit configured to generate a prediction image from the interpolation image, wherein
- in a case that in the weighted prediction processing unit, a bi-prediction is performed and information of the weight coefficient and the offset value is absent for both a reference list 0 and a reference list 1, the normal prediction processing unit generates a bi-prediction image.
- A video decoding apparatus according to an aspect of the present invention includes:

- a weighted prediction processing unit configured to decode a weight coefficient and an offset value from coded data and to generate a prediction image by multiplying an interpolation image by the weight coefficient and adding the offset value to the interpolation image, and

3

a normal prediction processing unit configured to generate a prediction image from the interpolation image, wherein

in a case that in the weighted prediction processing unit, uni-prediction (I.0 prediction or L.1 prediction) is performed and information of the weight coefficient and the offset value for the reference list is absent, the normal prediction processing unit generates a uni-prediction image.

A video coding apparatus according to an aspect of the present invention includes:

a weighted prediction processing unit configured to code a weight coefficient and an offset value into coded data and to generate a prediction image by multiplying an interpolation image by the weight coefficient and adding the offset value to the interpolation image, and

a normal prediction processing unit configured to generate a prediction image from the interpolation image, wherein

in a case that in the weighted prediction processing unit, a bi-prediction is performed and information of the weight coefficient and the offset value is absent for both a reference list 0 and a reference list 1, the normal prediction processing unit generates a bi-prediction image.

A video coding apparatus according to an aspect of the present invention includes:

a weighted prediction processing unit configured to code a weight coefficient and an offset value into coded data and to generate a prediction image by multiplying an interpolation image by the weight coefficient and adding the offset value to the interpolation image, and

a normal prediction processing unit configured to generate a prediction image from the interpolation image, wherein

in a case that in the weighted prediction processing unit, a uni-prediction is performed and information of the weight coefficient and the offset value for the reference list is absent, the normal prediction processing unit generates a uni-prediction image.

Advantageous Effects of Invention

According to an aspect of the present invention, it is possible to specify separately for luminance and chrominance signals that the weighted prediction is not performed in a case that no weight coefficient is present in video coding and decoding processing, allowing the above-described problem to be solved.

BRIEF DESCRIPTION OF DRAWINGS

FIG. 1 is a schematic diagram illustrating a configuration of an image transmission system according to the present embodiment.

FIG. 2 is a diagram illustrating configurations of a transmitting apparatus equipped with a video coding apparatus and a receiving apparatus equipped with a video decoding apparatus according to the present embodiment. PROD_A illustrates the transmitting apparatus equipped with the video coding apparatus, and PROD_B illustrates the receiving apparatus equipped with the video decoding apparatus.

FIG. 3 is a diagram illustrating configurations of a recording apparatus equipped with the video coding apparatus and a reconstruction apparatus equipped with the video decoding apparatus according to the present embodiment. PROD_C illustrates the recording apparatus equipped with the video

4

coding apparatus, and PROD_D illustrates the reconstruction apparatus equipped with the video decoding apparatus.

FIG. 4 is a diagram illustrating a hierarchical structure of data of a coding stream.

FIG. 5 is a conceptual diagram of an image to be processed in the image transmission system according to the present embodiment.

FIG. 6 is a conceptual diagram illustrating an example of reference pictures and reference picture lists.

FIG. 7 is a schematic diagram illustrating a configuration of a video decoding apparatus.

FIG. 8 is a flowchart illustrating general operation of the video decoding apparatus.

FIG. 9 is a diagram illustrating a configuration example of a syntax table of slice headers.

FIG. 10 is a flowchart illustrating an example of processing of the present application.

FIG. 11 is a schematic diagram illustrating a configuration of an inter prediction parameter derivation unit.

FIG. 12 is a schematic diagram illustrating a configuration of a merge prediction parameter derivation unit and an AMVP prediction parameter derivation unit.

FIG. 13 is a diagram illustrating the motion vector $spMvLX[xi][yi]$ of affine prediction.

FIG. 14 is a schematic diagram illustrating a configuration of an inter prediction image generation unit.

FIG. 15 is a table related to derivation of coefficients in Interpolation Filter processing.

FIG. 16 is a table related to the derivation of the coefficients in the Interpolation Filter processing.

FIG. 17 is a table related to the derivation of the coefficients in the Interpolation Filter processing.

FIG. 18 is a table related to the derivation of the coefficients in the Interpolation Filter processing.

FIG. 19 is a table related to the derivation of the coefficients in the Interpolation Filter processing.

FIG. 20 is a table related to the derivation of the coefficients in the Interpolation Filter processing.

FIG. 21 is a table related to the derivation of the coefficients in the Interpolation Filter processing.

FIG. 22 is a block diagram illustrating a configuration of a video coding apparatus.

FIG. 23 is a schematic diagram illustrating a configuration of an inter prediction parameter coder.

FIG. 24 is a schematic diagram illustrating a configuration of an intra prediction parameter coder.

FIG. 25 is a flowchart illustrating a flow of processing in which a BDOF unit derives a prediction image.

FIG. 26 is a schematic diagram illustrating a configuration of the BDOF unit.

FIG. 27 is a diagram illustrating a syntax of coding parameters for a weighted prediction.

FIG. 28 is a diagram illustrating a syntax of coding parameters for a weighted prediction.

FIG. 29 is a diagram illustrating a syntax of a BCW prediction.

FIG. 30 is a diagram illustrating a syntax of coding parameters for a weighted prediction.

DESCRIPTION OF EMBODIMENTS

First Embodiment

Hereinafter, an embodiment of the present invention will be described with reference to the drawings.

5

FIG. 1 is a schematic diagram illustrating a configuration of an image transmission system 1 according to the present embodiment.

The image transmission system 1 is a system for transmitting the coding stream in which an image of different resolution transformed in resolution is coded, decoding the coding stream transmitted, and inversely transforming the coding stream decoded into the image with the original resolution for display. The image transmission system 1 includes a resolution transform apparatus (resolution transform unit) 51, a video coding apparatus (image coding apparatus) 11, a network 21, a video decoding apparatus (image decoding apparatus) 31, a resolution inverse transform apparatus (resolution inverse transform unit) 61, and a video display apparatus (image display apparatus) 41.

The resolution transform apparatus 51 transforms the resolution of an image T included in a video, and supplies a variable resolution video signal including the image with a different resolution to the image coding apparatus 11. The resolution transform apparatus 51 supplies, to the video coding apparatus 11, information indicating the presence or absence of resolution transform of the image. In a case that the information indicates resolution transform, the video coding apparatus sets the resolution transform information ref_pic_resampling_enabled_flag described below to 1, and includes the information in a sequence parameter set SPS (SequenceParameter Set) of coded data for coding.

The image T with the transformed resolution is input to the video coding apparatus 11.

The network 21 transmits a coding stream Te generated by the video coding apparatus 11 to the video decoding apparatus 31. The network 21 is the Internet, a Wide Area Network (WAN), a Local Area Network (LAN), or a combination thereof. The network 21 is not necessarily limited to a bidirectional communication network, and may be a unidirectional communication network configured to transmit broadcast waves of digital terrestrial television broadcasting, satellite broadcasting of the like. Furthermore, the network 21 may be substituted by a storage medium in which the coding stream Te is recorded, such as a Digital Versatile Disc (DVD: trade name) or a Blue-ray Disc (BD: trade name).

The video decoding apparatus 31 decodes each of the coding streams Te transmitted by the network 21 and generates and supplies a variable resolution decoded image signal to the resolution inverse transform apparatus 61.

In a case that the resolution transform information included in the variable resolution decoded image signal indicates resolution transform, the resolution inverse transform apparatus 61 generates a decoded image signal with the original size by inversely transforming the resolution-transformed image.

The video display apparatus 41 displays all or part of one or multiple decoded images Td indicated by the decoded image signal received from the resolution inverse transform unit. For example, the video display apparatus 41 includes a display device such as a liquid crystal display and an organic Electro-Luminescence (EL) display. Forms of the display include a stationary type, a mobile type, an HMD type, and the like. In addition, in a case that the video decoding apparatus 31 has a high processing capability, an image having high image quality is displayed, and in a case that the apparatus has a lower processing capability, an image which does not require high processing capability and display capability is displayed.

FIG. 5 is a conceptual diagram of an image to be processed in the image transmission system 1 illustrated in

6

FIG. 1, and is a diagram illustrating a change in resolution of the image over time. Note that, FIG. 5 does not distinguish whether the image is coded. FIG. 5 illustrates an example in which, during the processing process of the image transmission system 1, an image with the resolution reduced is transmitted to the image decoding apparatus 31. As illustrated in FIG. 5, typically, the resolution transform apparatus 51 performs a transform for reducing the resolution of the image to decrease the amount of information to be transmitted.

Operator

Operators used in the present specification will be described below.

>> is a right bit shift, << is a left bit shift, & is a bitwise AND, | is a bitwise OR, |= is an OR assignment operator, and || indicates a logical sum.

x?y: z is a ternary operator that takes y in a case that x is true (other than 0) and takes z in a case that x is false (0).

Clip3(a, b, c) is a function to clip c in a value equal to or greater than a and less than or equal to b, and a function to return a in a case that c is less than a (c<a), return b in a case that c is greater than b (c>b), and return c in other cases (provided that a is less than or equal to b (a<=b)).

abs(a) is a function that returns the absolute value of a.

Int(a) is a function that returns the integer value of a.

floor(a) is a function that returns the maximum integer equal to or less than a.

ceil(a) is a function that returns the minimum integer equal to or greater than a.

a/d represents division of a by d (round down decimal places).

Structure of Coding Stream Te

Prior to the detailed description of the video coding apparatus 11 and the video decoding apparatus 31 according to the present embodiment, a data structure of the coding stream Te generated by the video coding apparatus 11 and decoded by the video decoding apparatus 31 will be described.

FIG. 4 is a diagram illustrating a hierarchical structure of data of the coding stream Te. The coding stream Te includes a sequence and multiple pictures constituting the sequence illustratively. FIG. 4 is a diagram illustrating a coded video sequence defining a sequence SEQ, a coded picture prescribing a picture PICT, a coding slice prescribing a slice S, a coding slice data prescribing slice data, a coding tree unit included in the coding slice data, and a coding unit included in the coding tree unit.

Coded Video Sequence

In the coded video sequence, a set of data referenced by the video decoding apparatus 31 to decode the sequence SEQ to be processed is defined. As illustrated in FIG. 4, the sequence SEQ includes a Video Parameter Set VPS, a Sequence Parameter Set SPS, a Picture Parameter Set PPS, an Adaptation Parameter Set (APS), a picture PICT, and Supplemental Enhancement Information SEI.

In the video parameter set VPS, in a video including multiple layers, a set of coding parameters common to multiple videos and a set of coding parameters associated with the multiple layers and an individual layer included in the video are defined.

In the sequence parameter set SPS, a set of coding parameters referenced by the video decoding apparatus 31 to decode a target sequence is defined. For example, a width

and a height of a picture are defined. Note that multiple SPSs may exist. In that case, any of the multiple SPSs is selected from the PPS.

Here, the sequence parameter set SPS includes the following syntax.

ref_pic_resampling_enabled_flag: A flag specifying whether to use a function of making the variable resolution (resampling) in a case of decoding images included in a single sequence referencing a target SPS. From another aspect, the flag indicates that the size of the reference picture, referenced in the generation of the prediction image, changes between the images indicated by the single sequence. In a case that the value of the flag is 1, the above resampling is applied, and in a case that the value is 0, the resampling is not applied.

pic_width_max_in_luma_samples: A syntax indicating, in units of luminance blocks, the width of one of the images in a single sequence, the image having the largest width. The syntax has a value that is not 0 and that is an integer multiple of Max(8, MinCbSizeY). Here, MinCbSizeY is a value determined by the minimum size of the luminance block.

pic_height_max_in_luma_samples: A syntax indicating, in units of luminance blocks, the height of one of the images in a single sequence, the image having the largest height. The syntax has a value that is not 0 and that is an integer multiple of Max(8, MinCbSizeY).

sps_temporal_mvp_enabled_flag: A flag specifying whether to use a temporal motion vector prediction in the case of decoding a target sequence. In a case that the value of the flag is 1, the temporal motion vector prediction is used, and in a case that the value is 0, the temporal motion vector prediction is not used. With this flag defined, in a case that reference pictures with different resolutions are referenced or in other such cases, coordinate positions to be referenced can be prevented from being misaligned.

In the picture parameter set PPS, a set of coding parameters referenced by the video decoding apparatus 31 to decode each picture in a target sequence is defined. For example, a reference value (pic_init_qp_minus26) of a quantization step size used for decoding of a picture and a flag (weighted_pred_flag) indicating an application of a weight prediction are included. Note that multiple PPSs may exist. In that case, any of the multiple PPSs is selected from each picture in a target sequence.

Here, the picture parameter set PPS includes the following syntax.

pic_width_in_luma_samples: A syntax indicating the width of a target picture. The syntax is required to have a value that is not 0 and that is an integer multiple of Max(8, MinCbSizeY) and that is equal to or less than pic_width_max_in_luma_samples.

pic_height_in_luma_samples: A syntax indicating the height of the target picture. The syntax is required to have a value that is not 0 and that is an integer multiple of Max(8, MinCbSizeY) and that is equal to or less than pic_height_max_in_luma_samples.

conformance_window_flag: A flag indicating whether a conformance (cropping) window offset parameter is subsequently signaled, the flag indicating a location where a conformance window is displayed. In a case that the flag is 1, the parameter is signaled, and in a case that the flag is 0, then no conformance window offset parameter is present.

conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, conf_win_bottom_offset: An offset value indicating, for a rectangular region indicated in picture coordinates for output, the left, right, top, and bottom positions of a picture output in decoding processing. In a case that the value of the conformance_window_flag is 0, the values of conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, conf_win_bottom_offset are inferred to be 0.

scaling_window_flag: A flag indicating whether a scaling window offset parameter is present in the target PPS, the flag being related to specification of an output image size. The flag being 1 indicates that the parameter is present in the PPS, and the flag being 0 indicates that the parameter is not present in the PPS. In a case that the value of ref_pic_resampling_enabled_flag is 0, then the value of scaling_window_flag is required to be 0.

scaling_win_left_offset, scaling_win_right_offset, scaling_win_top_offset, scaling_win_bottom_offset: A syntax indicating an offset applied to the image size for scaling ratio calculation, in luminance sample units for the left, right, top, and bottom positions of the target picture. In a case that the value of the scaling_window_flag is 0, the values of scaling_win_left_offset, scaling_win_right_offset, scaling_win_top_offset, scaling_win_bottom_offset are inferred to be 0. The value of scaling_win_left_offset+scaling_win_right_offset is required to be less than pic_width_in_luma_samples, and the value of scaling_win_top_offset+scaling_win_bottom_offset is required to be less than pic_height_in_luma_samples.

The width PicOutputWidthL and the height PicOutputHeightL of the output picture are derived as described below.

$$\begin{aligned} \text{PicOutputWidthL} &= \text{pic_width_in_luma_samples} - \\ &(\text{scaling_win_right_offset} + \text{scaling_win_left_offset}) \\ \text{PicOutputHeightL} &= \text{pic_height_in_pic_size_units} - \\ &(\text{scaling_win_bottom_offset} + \text{scaling_win_top_offset}) \end{aligned}$$

pps_collocated_from_10_idc: A syntax indicating whether collocated_from_10_flag is present in a slice header of a slice referencing the PPS. The value of the syntax being 0 indicates that collocated_from_10_flag is present in the slice header. The value of the syntax being 1 or 2 indicates that collocated_from_10_flag is not present in the slice header.

Coded Picture

In the coded picture, a set of data referenced by the video decoding apparatus 31 to decode the picture PICT to be processed is defined. As illustrated in FIG. 4, the picture PICT includes a picture header PH and slices 0 to NS-1 (NS is the total number of slices included in the picture PICT).

In the description below, in a case that the slices 0 to NS-1 need not be distinguished from one another, subscripts of reference signs may be omitted. In addition, the same applies to other data with subscripts included in the coding stream Te which will be described below.

The picture header includes the following syntax.

pic_temporal_mvp_enabled_flag: A flag specifying whether a temporal motion vector prediction is used for an inter prediction of a slice associated with the picture header. In a case that the value of the flag is 0, the

syntax element of a slice associated with the picture header is restricted such that the temporal motion vector prediction is not used to decode the slice. The value of the flag being 1 indicates that the temporal motion vector prediction is used to decode the slice associated with the picture header. In a case that the flag is not specified, the value is inferred to be 0.

Coding Slice

In the coding slice, a set of data referenced by the video decoding apparatus 31 to decode the slice S to be processed is defined. As illustrated in FIG. 4, the slice includes a slice header and slice data.

The slice header includes a coding parameter group referenced by the video decoding apparatus 31 to determine a decoding method for a target slice. Slice type specification information (slice_type) indicating a slice type is one example of a coding parameter included in the slice header.

Examples of slice types that can be indicated by the slice type indication information include (1) I slices for which only an intra prediction is used in coding, (2) P slices for which a uni-prediction (L0 prediction) or an intra prediction is used in coding, and (3) B slices for which a uni-prediction (L0 prediction or L1 prediction), a bi-prediction, or an intra prediction is used in coding, and the like. Note that the inter prediction is not limited to a uni-prediction and a bi-prediction, and the prediction image may be generated by using a larger number of reference pictures. Hereinafter, in a case of being referred to as the P or B slice, a slice that includes a block in which the inter prediction can be used is indicated.

Note that the slice header may include a reference to the picture parameter set PPS (pic_parameter_set_id).

The slice header may include the following syntax as illustrated in FIG. 9.

collocated_from_10_flag: A flag specifying which reference picture list a collocated picture used for a temporal motion vector prediction is derived from. The value of the flag being 1 indicates that the picture is derived from the reference picture list 0 (L0 list), and the value of the flag being 0 indicates that the picture is derived from the reference picture list 1 (L1 list). In a case that the flag is not specified, a value of 0 or 1 is inferred depending on the type of slice and pps_collocated_from_10_idc.

In a case that the slice type is other than B, collocated_from_10_flag is inferred to be 1. In a case that the slice type is B, collocated_from_10_flag is inferred to be pps_collocated_from_10_idc-1.

collocated_ref_idx: A reference index of a collocated picture used for a temporal motion vector prediction. The value of collocated_ref_idx is specified by the type of slice or the value of collocated_from_10_flag. In a case that the slice type is P, or alternatively, the slice type is B and collocated_from_10_flag is 1, then collocated_ref_idx references a picture in List0. In a case that the slice type is B and collocated_from_10_flag is 0, collocated_ref_idx references a picture of List1. In a case that collocated_ref_idx is absent, then the flag is inferred to be 0.

In the present embodiment, in a case that the variable RefPicIsScaled [collocated_from_10_flag?0:1][collocated_ref_idx], indicating that the size of a coded picture is different from the size of a reference picture, has a value of 1, the variable Slice TemporalMvpEnableFlag is set equal to 0. Otherwise, the variable Slice TemporalMvpEnableFlag is set equal to the value of pic_temporal_mvp_enabled_flag.

Thus, in a case that a reference picture with a size different from the size of the coded picture is indicated by collocated_ref_idx, the variable SliceTemporalMvpEnableFlag is forcibly set equal to 0, and coding and decoding processing is performed on the slice without the use of the motion vector information of the reference picture.

FIG. 10 illustrates the above-described processing in a flowchart. First, the variable X is set equal to the value of collocated_from_10_flag corresponding to a flag specifying which reference picture list L0 or L1 a collocated picture, collocated and used for a temporal motion vector prediction, is derived from. (S201) Then, whether the value of the variable RefPicIsScaled is 1 is determined in order to examine whether the size of the collocated picture, indicated in collocated_ref_idx is different from the size of the coded picture. (S202) In a case that the value of the variable RefPicIsScaled is 1, the size of the collocated picture is determined to be different from the size of the coded picture. The variable SliceTemporalMvpEnableFlag is set equal to 0, and the motion vector information of the collocated picture is not used. (S203) In a case that the value of the variable RefPicIsScaled is not 1, the variable SliceTemporalMvpEnableFlag is set equal to the value of picTemporalMvpEnableFlag corresponding to a flag at the picture level. (S204)

In another embodiment, in a case that a reference picture with a size different from the size of the coded picture is specified in collocated_ref_idx, the smallest value of ref_idx of the reference pictures with the same size as that of the coded picture may be specified as collocated_ref_idx. In a case that none of the reference pictures has the same size as that of the coded picture, the variable SliceTemporalMvpEnableFlag may be forcibly set equal to 0, and coding and decoding processing may be performed on the slice without the use of the motion vector information of the reference picture.

In any event, in a case that a reference picture with a size different from that of the coded picture is specified in collocated_ref_idx, then a procedure is set in which the decoding processing always includes the same operations.

In this way, by defining the processing performed in a case that a reference picture with a size different from the size of the coded picture is specified in collocated_ref_idx instead of prohibiting generation of coded data indicating, in collocated_ref_idx, a reference picture with a size different from the size of the coded picture, the decoding processing can be guaranteed to always include the same operations.

Coding Slice Data

In the coding slice data, a set of data referenced by the video decoding apparatus 31 to decode the slice data to be processed is defined. The slice data includes CTUs as illustrated in the coding slice header in FIG. 4. The CTU is a block of a fixed size (for example, 64×64) constituting a slice, and may be called a Largest Coding Unit (LCU).

Coding Tree Unit

In FIG. 4, a set of data is defined that is referenced by the video decoding apparatus 31 to decode the CTU to be processed. The CTU is split into coding units CUs, each of which is a basic unit of coding processing, by a recursive Quad Tree split (QT split), Binary Tree split (BT split), or Ternary Tree split (TT split). The BT split and the TT split are collectively referred to as a Multi Tree split (MT split). Nodes of a tree structure obtained by recursive quad tree splits are referred to as Coding Nodes. Intermediate nodes of a quad tree, a binary tree, and a ternary tree are coding nodes, and the CTU itself is also defined as the highest coding node.

11

The CT includes, as CT information, a CU split flag (split_cu_flag) indicating whether or not to perform a CT split, a QT split flag (qt_split_cu_flag) indicating whether or not to perform a QT split, an MT split flag (mtt_split_cu_flag) indicating the presence or absence of an MT split, an MT split direction (mtt_split_cu_vertical_flag) indicating a split direction of an MT split, and an MT split type (mtt_split_cu_binary_flag) indicating a split type of the MT split. split_cu_flag, qt_split_cu_flag, mtt_split_cu_flag, mtt_split_cu_vertical_flag, and mtt_split_cu_binary_flag are transmitted for each coding node.

Different trees may be used between luma and chroma. The type of the tree is represented by treeType. For example, in a case that a common tree is used for luminance (Y, cldx=0) and chrominance (Cb/Cr, cldx=1,2), a common single tree is represented by treeType=SINGLE_TREE. In a case that two different trees (DUAL tree) are used for luminance and chrominance, the tree of luma is represented by treeType=DUAL_TREE_LUMA, and the tree of chrominance is represented by treeType=DUAL_TREE_CHROMA.

Coding Unit

In FIG. 4, a set of data referenced by the video decoding apparatus 31 to decode the coding unit to be processed is defined. Specifically, the CU includes a CU header CUH, a prediction parameter, a transform parameter, a quantization transform coefficient, and the like. In the CU header, a prediction mode and the like are defined.

There are cases that the prediction processing is performed in units of CU or performed in units of sub-CU in which the CU is further split. In a case that the sizes of the CU and the sub-CU are equal to each other, the number of sub-CUs in the CU is one. In a case that the CU is larger in size than the sub-CU, the CU is split into sub-CUs. For example, in a case that the CU has a size of 8×8, and the sub-CU has a size of 4×4, the CU is split into four sub-CUs which include two horizontal splits and two vertical splits.

There are two types of predictions (prediction modes), which are intra prediction and inter prediction. The intra prediction refers to a prediction in an identical picture, and the inter prediction refers to prediction processing performed between different pictures (for example, between pictures of different display times, and between pictures of different layer images).

Transform and quantization processing is performed in units of CU, but the quantization transform coefficient may be subjected to entropy coding in units of subblock such as 4×4.

Prediction Parameter

A prediction image is derived by prediction parameters accompanying a block. The prediction parameters include prediction parameters for intra prediction and inter prediction.

The prediction parameters for inter prediction will be described below. The inter prediction parameters include prediction list utilization flags predFlagL0 and predFlagL1, reference picture indexes reflDxL0 and reflDxL1, and motion vectors mvL0 and mvL1. predFlagL0 and predFlagL1 are flags indicating whether reference picture lists (L0 list and L1 list) are used, and in a case that the value of each of the flags is 1, a corresponding reference picture list is used. Note that, in a case that the present specification mentions “a flag indicating whether or not XX”, a flag being other than 0 (for example, 1) assumes a case of XX, and a flag being 0 assumes a case of not XX, and 1 is treated as true and 0 is treated as false in a logical negation, a logical product, and the like (hereinafter, the same is applied).

12

However, other values can be used for true values and false values in real apparatuses and methods.

For example, syntax elements to derive the inter prediction parameters include an affine flag affine_flag, a merge flag merge_flag, a merge index merge_idx, and an MMVD flag mmvd_flag that are used in the merge mode, an inter prediction indicator inter_pred_idc and a reference picture index reflDxLX that are used to select a reference picture in the AMVP mode, a prediction vector index mvp_LX_idx, a difference vector mvD_LX, and a motion vector precision mode amvr_mode that are used to derive a motion vector. Reference Picture List

A reference picture list is a list including reference pictures stored in a reference picture memory 306. FIG. 6 is a conceptual diagram illustrating an example of reference pictures and reference picture lists. In FIG. 6 corresponding to a conceptual diagram illustrating an example of reference pictures, rectangles indicate pictures, arrows indicate reference relationships among the pictures, a horizontal axis indicates time, I, P, and B in the rectangles respectively indicate an intra-picture, a uni-prediction picture, and a bi-prediction picture, and numbers in the rectangles indicate a decoding order. As illustrated, the decoding order of the pictures is I0, P1, B2, B3, and B4, and the display order is I0, B3, B2, B4, and P1. FIG. 6 illustrates an example of reference picture list of the picture B3 (target picture). The reference picture list is a list to represent a candidate of a reference picture, and one picture (slice) may include one or more reference picture lists. In the illustrated example, the target picture B3 includes two reference picture lists, i.e., an L0 list RefPicList0 and an L1 list RefPicList1. For individual CUs, which picture in a reference picture list RefPicListX (X=0 or 1) is actually referenced is specified with reflDxLX. The diagram illustrates an example of reflDxL0=2, reflDxL1=0. Note that LX is a description method used in a case of not distinguishing an L0 prediction and an L1 prediction, and in the following description, distinguishes parameters for the L0 list and parameters for the L1 list by replacing LX with L0 and L1.

Merge Prediction and AMVP Prediction

A decoding (coding) method for prediction parameters include a merge prediction (merge) mode and an Advanced Motion Vector Prediction (AMVP) mode, and merge_flag is a flag to identify the modes. The merge prediction mode is a mode in which a prediction list utilization flag predFlagLX, the reference picture index reflDxLX, and a motion vector mvLX are derived from prediction parameters for neighboring blocks already processed, or the like, without being included in the coded data. The AMVP mode is a mode in which inter_pred_idc, reflDxLX, and mvLX are included in the coded data. Note that, mvLX is coded as mvp_LX_idx identifying a prediction vector mvpLX and a difference vector mvD_LX. In addition to the merge prediction mode, an affine prediction mode and an MMVD prediction mode may be available.

inter_pred_idc is a value indicating the types and number of reference pictures, and takes any value of PRED_L0, PRED_L1, or PRED_BI. PRED_L0 and PRED_L1 indicate uni-predictions which use one reference picture managed in the L0 list and one reference picture managed in the L1 list, respectively. PRED_BI indicates a bi-prediction which uses two reference pictures managed in the L0 list and the L1 list.

merge_idx is an index to indicate which prediction parameter is used as a prediction parameter for the target

13

block, among prediction parameter candidates (merge candidates) derived from blocks of which the processing is completed.

Motion Vector

mvLX indicates a shift amount between blocks in two different pictures. A prediction vector and a difference vector related to mvLX are respectively referred to as mvpLX and mvdLX.

Inter Prediction Indicator *inter_pred_idc* and Prediction List Utilization Flag *predFlagLX*

Relationships between *inter_pred_idc* and *predFlagL0* and *predFlagL1* are as follows, and can be transformed into one another.

$$\begin{aligned} \text{inter_pred_idc} &= (\text{predFlagL1} \ll 1) + \text{predFlagL0} \\ \text{predFlagL0} &= \text{inter_pred_idc} \& 1 \\ \text{predFlagL1} &= \text{inter_pred_idc} \gg 1 \end{aligned}$$

Note that the inter prediction parameters may use a prediction list utilization flag or may use an inter prediction indicator. A determination using a prediction list utilization flag may be replaced with a determination using an inter prediction indicator. On the contrary, a determination using an inter prediction indicator may be replaced with a determination using a prediction list utilization flag.

Determination of Bi-Prediction *biPred*

A flag *biPred* for identifying a bi-prediction can be derived from whether two prediction list utilization flags are both 1. For example, the derivation can be performed by the following equation.

$$\text{biPred} = (\text{predFlagL0} == 1 \&\& \text{predFlagL1} == 1)$$

Alternatively, *biPred* can be also derived from whether the inter prediction indicator is a value indicating the use of two prediction lists (reference pictures). For example, the derivation can be performed by the following equation.

$$\text{biPred} = (\text{inter_pred_idc} == \text{PRED_BI}) ? 1 : 0$$

Configuration of Video Decoding Apparatus

The configuration of the video decoding apparatus **31** (FIG. 7) according to the present embodiment will be described.

The video decoding apparatus **31** includes an entropy decoder **301**, a parameter decoder (a prediction image decoding apparatus) **302**, a loop filter **305**, a reference picture memory **306**, a prediction parameter memory **307**, a prediction image generation unit (prediction image generation apparatus) **308**, an inverse quantization and inverse transform processing unit **311**, an addition unit **312**, and a prediction parameter derivation unit **320**. Note that a configuration in which the loop filter **305** is not included in the video decoding apparatus **31** may be used in accordance with the video coding apparatus **11** described later.

The parameter decoder **302** further includes a header decoder **3020**, a CT information decoder **3021**, and a CU decoder **3022** (prediction mode decoder), and the CU decoder **3022** further includes a TU decoder **3024**. These may be collectively referred to as a decoding module. The

14

header decoder **3020** decodes, from coded data, parameter set information such as the VPS, the SPS, the PPS, and an APS, and a slice header (slice information). The CT information decoder **3021** decodes a CT from coded data. The CU decoder **3022** decodes a CU from coded data. In a case that a TU includes a prediction error, the TU decoder **3024** decodes QP update information (quantization correction value) and quantization prediction error (residual_coding) from coded data.

In the mode other than the skip mode (*skip_mode*=0), the TU decoder **3024** decodes QP update information and quantization prediction error from coded data. More specifically, the TU decoder **3024** decodes, in a case of *skip_mode*=0, a flag *cu_cbp* indicating whether a quantization prediction error is included in the target block, and decodes the quantization prediction error in a case that *cu_cbp* is 1. In a case that *cu_cbp* is not present in the coded data, the TU decoder **3024** derives *cu_cbp* as 0.

The TU decoder **3024** decodes an index *mts_idx* indicating a transform basis from the coded data. The TU decoder **3024** decodes, from the coded data, an index *stIdx* indicating the use of a secondary transformation and the transform basis. *stIdx* being 0 indicates non-application of the secondary transformation, *stIdx* being 1 indicates transformation of one of a set (pair) of secondary transform bases, and *stIdx* being 2 indicates transformation of the other of the pair of secondary transform bases.

The TU decoder **3024** may decode a subblock transformation flag *cu_sbt_flag*. In a case that *cu_sbt_flag* is 1, the CU is split into multiple subblocks, and for only one particular subblock, the residual is decoded. Furthermore, the TU decoder **3024** may decode the flag *cu_sbt_quad_flag* indicating whether the number of subblocks is 4 or 2, *cu_sbt_horizontal_flag* indicating a split direction, and *cu_sbt_pos_flag* indicating a subblock including a non-zero transform coefficient.

The prediction image generation unit **308** includes an inter prediction image generation unit **309** and an intra prediction image generation unit **310**.

The prediction parameter derivation unit **320** includes an inter prediction parameter derivation unit **303** and an intra prediction parameter derivation unit **304**.

Furthermore, an example in which a CTU and a CU are used as units of processing is described below, but the processing is not limited to this example, and processing in units of sub-CU may be performed. Alternatively, the CTU and the CU may be replaced with a block, the sub-CU may be replaced with by a subblock, and processing may be performed in units of blocks or subblocks.

The entropy decoder **301** performs entropy decoding on the coding stream *Te* input from the outside and separates and decodes individual codes (syntax elements). The entropy coding includes a scheme in which syntax elements are subjected to variable length coding by using a context (probability model) that is adaptively selected according to a type of the syntax elements and a surrounding condition, and a scheme in which syntax elements are subjected to variable length coding by using a table or a calculation expression that is determined in advance. The former CABAC (Context Adaptive Binary Arithmetic Coding) stores in memory a CABAC state of the context (the type of a dominant symbol (0 or 1) and a probability state index *pStateIdx* indicating a probability). The entropy decoder **301** initializes all CABAC states at the beginning of a segment (tile, CTU row, or slice). The entropy decoder **301** transforms the syntax element into a binary string (Bin String) and decodes each bit of the Bin String. In a case that the

15

context is used, a context index *ctxInc* is derived for each bit of the syntax element, the bit is decoded using the context, and the CABAC state of the context used is updated. Bits that do not use the context are decoded at an equal probability (EP, bypass), and the *ctxInc* derivation and CABAC state are omitted. The decoded syntax element includes prediction information for generating a prediction image, a prediction error for generating a difference image, and the like.

The entropy decoder **301** outputs the decoded codes to the parameter decoder **302**. The decoded code is, for example, a prediction mode *predMode*, *merge_flag*, *merge_idx*, *inter_pred_idc*, *refLdX*, *mVP_Lx_idx*, *mvdLX*, *amvr_mode*, and the like. Which code is to be decoded is controlled based on an indication of the parameter decoder **302**.

Basic Flow of Operation

FIG. 8 is a flowchart for describing general operation performed in the video decoding apparatus **31**.

(**S1100**: Decoding of parameter set information) The header decoder **3020** decodes parameter set information such as the VPS, the SPS, and the PPS from coded data.

(**S1200**: Decoding of slice information) The header decoder **3020** decodes a slice header (slice information) from the coded data.

Afterwards, the video decoding apparatus **31** repeats the processing from **S1300** to **S5000** for each CTU included in the target picture, and thereby derives a decoded image of each CTU.

(**S1300**: Decoding of CTU information) The CT information decoder **3021** decodes the CTU from the coded data.

(**S1400**: Decoding of CT information) The CT information decoder **3021** decodes the CT from the coded data.

(**S1500**: Decoding of CU) The CU decoder **3022** decodes the CU from the coded data by performing **S1510** and **S1520**.

(**S1510**: Decoding of CU information) The CU decoder **3022** decodes, for example, CU information, prediction information, a TU split flag *split_transform_flag*, CU residual flags *cbf_cb*, *cbf_cr*, and *cbf_luma* from the coded data.

(**S1520**: Decoding of TU information) In a case that a prediction error is included in the TU, the TU decoder **3024** decodes, from the coded data, QP update information and a quantization prediction error, and transform index *mts_idx*. Note that the QP update information is a difference value from a quantization parameter prediction value *qPpred*, which is a prediction value of a quantization parameter QP.

(**S2000**: Generation of prediction image) The prediction image generation unit **308** generates a prediction image, based on the prediction information, for each block included in the target CU.

(**S3000**: Inverse quantization and inverse transform) The inverse quantization and inverse transform processing unit **311** performs inverse quantization and inverse transform processing on each TU included in the target CU.

(**S4000**: Generation of decoded image) The addition unit **312** generates a decoded image of the target CU by adding the prediction image supplied by the prediction image generation unit **308** and the prediction error supplied by the inverse quantization and inverse transform processing unit **311**.

(**S5000**: Loop filter) The loop filter **305** generates a decoded image by applying a loop filter such as a deblocking filter, an SAO, and an ALF to the decoded image.

Configuration of Inter Prediction Parameter Derivation Unit

16

FIG. 11 is a schematic diagram illustrating a configuration of the inter prediction parameter derivation unit **303** according to the present embodiment. The inter prediction parameter derivation unit **303** derives an inter prediction parameter with reference to the prediction parameters stored in the prediction parameter memory **307**, based on the syntax element input from the parameter decoder **302**. The inter prediction parameter derivation unit **303** outputs the inter prediction parameter to the inter prediction image generation unit **309** and the prediction parameter memory **307**. The following are means shared by the video coding apparatus and the video decoding apparatus, and may thus be collectively referred to as a motion vector derivation unit (motion vector derivation apparatus): the inter prediction parameter derivation unit **303** and the internal elements of the inter prediction parameter derivation unit **303** including an AMVP prediction parameter derivation unit **3032**, a merge prediction parameter derivation unit **3036**, an affine prediction unit **30372**, and an MMVD prediction unit **30373**, a triangle prediction unit **30377**, a DMVR unit **30537**, and an MV addition unit **3038**.

The scale parameter derivation unit **30378** derives the scaling ratio in the horizontal direction of the reference picture *RefPicScale[i][j][0]*, the scaling ratio in the vertical direction of the reference picture *RefPicScale[i][j][1]*, and *RefPicIsScaled[i][j]* indicating whether the reference picture is scaled. Here, with *i* indicating whether the reference picture list is an L0 list or an L1 list, and *j* being the value of the L0 reference picture list or the L1 reference picture list, the derivation is performed as follows.

$$RefPicScale[i][j][0] =$$

$$((fRefWidth \ll 14) + (PicOutputWidthL \gg 1)) / PicOutputWidthL$$

$$RefPicScale[i][j][1] =$$

$$((fRefHeight \ll 14) + (PicOutputHeightL \gg 1)) / PicOutputHeightL$$

$$RefPicIsScaled[i][j] =$$

$$(RefPicScale[i][j][0] \neq (1 \ll 14)) \vee (RefPicScale[i][j][1] \neq (1 \ll 14))$$

Here, the variable *PicOutputWidthL* is a value to calculate the scaling ratio in the horizontal direction when the coded picture is referenced, and is obtained by subtracting a left offset value and a right offset value from the number of pixels in the horizontal direction of the luminance of the coded picture. The variable *PicOutputHeightL* is a value to calculate the scaling ratio in the vertical direction when the coded picture is referenced, and is obtained by subtracting a top offset value and a bottom offset value from the number of pixels in the vertical direction of the luminance of the coded picture. The variable *fRefWidth* is the value of *PicOutputWidthL* of the reference picture of the reference list value *j* in the list *i*, and the variable *fRefHeight* is the value of *PicOutputHeightL* of the reference picture of the reference picture list value *j* in the list *i*.

In a case that the *affine_flag* indicates 1, that is, the affine prediction mode, the affine prediction unit **30372** derives the inter prediction parameters in subblock units.

In a case that the *mmvd_flag* indicates 1, that is, the MMVD prediction mode, the MMVD prediction unit **30373** derives an inter prediction parameter from the merge candidate and the difference vector derived by the merge prediction parameter derivation unit **3036**.

17

In a case that TriangleFlag indicates 1, that is, a triangle prediction mode, the triangle prediction unit **30377** derives a triangle prediction parameter.

In a case that merge_flag indicates 1, that is, the merge prediction mode, merge_idx is derived and output to the merge prediction parameter derivation unit **3036**.

In a case that the merge_flag indicates 0, that is, the AMVP prediction mode, the AMVP prediction parameter derivation unit **3032** derives mvplX from inter_pred_idc, refIdxLX, or mvpLx_idx.

MV Addition Unit

In the MV addition unit **3038**, mvplX and mvdLX derived are added together to derive mvLX.

Affine Prediction Unit

The affine prediction unit **30372** derives 1) motion vectors for two control points CP0, CP1 or three control points CP0, CP1, CP2 of the target block, 2) derive affine prediction parameters for the target block, and 3) derives a motion vector for each subblock from the affine prediction parameter.

In the case of merge affine prediction, a motion vector cpMvLX[] for each control point CP0, CP1, CP2 is derived from a motion vector for an adjacent block of the target block. In the case of inter affine prediction, cpMvLX[] for each control point is derived from the sum of the prediction vector for each control point CP0, CP1, CP2 and the difference vector mvdCpLX[] derived from the coded data.

FIG. 13 is a diagram illustrating an example in which a motion vector spMvLX of each subblock constituting a target block (bW*bH) is derived from a motion vector cpMvLX[0] of the control point CP0, a motion vector cpMvLX[1] of the control point CP1, and a motion vector cpMvLX[2] of the control point CP2. spMvLX is derived as a motion vector of a point (xPosCb, yPosCb) located at the center of each subblock, as illustrated.

The affine prediction unit **30372** derives an affine prediction parameter (mvScaleHor, mvScalerVer, dHorX, dHorY, dVerX, dVerY) for the target block from the motion vector for the control point.

```

mvScaleHor = cpMvLX[0][0] << 7
mvScaleVer = cpMvLX[0][1] << 7
dHorX = (cpMvLX[1][0] - cpMvLX[0][0]) << (7 - log2CbW)
dVerX = (cpMvLX[1][1] - cpMvLX[0][1]) << (7 - log2CbW)
for numCpMv = 3,
dHorY = (cpMvLX[2][0] - cpMvLX[0][0]) << (7 - log2CbH)
dVerY = (cpMvLX[2][1] - cpMvLX[0][1]) << (7 - log2CbH)
for numCpMv! = 3
dHorY = - dVerX
dVerY = dHorX
Here, log2CbW and log2CbH are logarithmic values of the width
bW and the height bH of the target block.

```

Based on the affine prediction parameter for the target block, the affine prediction unit **30372** derives spMvLX[i][j] in the target block (i=0, 1, 2, . . . , (bW/sbW)-1, j=0, 1, 2, . . . , (bH/sbH)-1) using the following equation.

$$xPosCb = 2 + (i \ll 2)$$

$$yPosCb = 2 + (j \ll 2)$$

$$spMvLX[i][j][0] = mvScaleHor + dHorX * xPosCb + dHorY * yPosCb$$

$$spMvLX[i][j][1] = mvScaleVer + dVerX * xPosCb + dVerY * yPosCb$$

18

Furthermore, round shifting and clipping may be performed.

$$spMvLX[i][j][0] = Clip3(-2^{17}, 2^{17}, (spMvLX[i][j][0] + 64) \gg 7)$$

$$spMvLX[i][j][1] = Clip3(-2^{17}, 2^{17}, (spMvLX[i][j][1] + 64) \gg 7)$$

Here, sbW and sbH are the width and height of the target subblock.

Furthermore, the coordinates (xSb, ySb) of the upper left block of the subblock are used to assign spMvLX[i][j] to mvLX in the corresponding picture. where x=0, . . . , sbW-1, and y=0, . . . , sbH-1.

$$mvLX[xSb + x][ySb + y] = spMvLX[i][j]$$

Merge Prediction

FIG. 12 is a schematic diagram illustrating a configuration of the merge prediction parameter derivation unit **3036** according to the present embodiment. The merge prediction parameter derivation unit **3036** includes a merge candidate derivation unit **30361** and a merge candidate selection unit **30362**. Note that a merge candidate includes the prediction parameter (predFlagLX, mvLX, and refIdxLX) and is stored in the merge candidate list. The merge candidate stored in the merge candidate list is assigned an index in accordance with a prescribed rule.

The merge candidate derivation unit **30361** derives the merge candidate using the motion vector and refIdxLX for the decoded adjacent block without any change. In addition, the merge candidate derivation unit **30361** may apply spatial merge candidate derivation processing, temporal merge candidate derivation processing, pairwise merge candidate derivation processing, and zero merge candidate derivation processing described below.

As the spatial merge candidate derivation processing, the merge candidate derivation unit **30361** reads the prediction parameter stored in the prediction parameter memory **307** in accordance with a prescribed rule, and configures the prediction parameter as a merge candidate. A reference picture specification method is, for example, a prediction parameter related to each of the adjacent blocks located within a prescribed range from the target block (e.g., all or some of a block 1 on the left of and share the boarder with the target block, a block B1 on the right of and share the boarder with the target block, a block B0 at the upper right of and tangent to the target block, a block A0 at the lower left of and tangent to the target block, and a block B2 at the upper left of and tangent to the target block). The merge candidates are referred to as A1, B1, B0, A0, and B2.

In this regard, A1, B1, B0, A0, and B2 are motion information derived from blocks including the following coordinates. In the target picture in FIG. 9, the positions of A1, B1, B0, A0, and B2 are illustrated in the arrangement of the merge candidates.

A1: (xCb - 1, yCb + cbHeight - 1)
 B1: (xCb + cbWidth - 1, yCb - 1)
 B0: (xCb + cbWidth, yCb - 1)
 A0: (xCb - 1, yCb + cbHeight)
 B2: (xCb - 1, yCb - 1)

The target block has upper left coordinates (xCb, yCb), a width cbWidth, and a height cbHeight.

19

As temporal merge derivation processing, the merge candidate derivation unit **30361** reads, from the prediction parameter memory **307**, the prediction parameter for a block C in the reference image including the lower right coordinates CBR or the center coordinates of the target block, specifies the block C as a merge candidate Col, and stores the block C in the merge candidate list `mergeCandList[ ]`, as illustrated in the collocated picture in FIG. 9.

In general, the block CBR is preferentially added to `mergeCandList[ ]`, and in a case that the CBR does not have a motion vector (e.g., an intra prediction block) or that the CBR is located outside the picture, then the motion vector of the block C is added to the prediction vector candidates. By adding, as a prediction candidate, a motion vector of a collocated block that is likely to move differently, the number of selections of the prediction vector is increased, enhancing coding efficiency.

In a case that `SliceTemporalMvpEnabledFlag` is 0 or `cbWidth*cbHeight` is less than or equal to 32, then a collocated motion vector `mvLXCcol` of the target block is set equal to 0, and an availability flag `availableFlagLXCcol` of the collocated block is set equal to 0.

Otherwise (in a case that `SliceTemporalMvpEnabledFlag` is 1), the following is performed.

For example, the merge candidate derivation unit **30361** may derive the position of the C (`xColCtr`, `yColCtr`) and the position of the CBR (`xColCBr`, `yColCBr`) by using the following equation.

$$xColCtr = xCb + (cbWidth \gg 1)$$

$$yColCtr = yCb + (cbHeight \gg 1)$$

$$xColCBr = xCb + cbWidth$$

$$yColCBr = yCb + cbHeight$$

In a case that the CBR is available, the motion vector of the CBR is utilized to derive a merge candidate COL. In a case that the CBR is not available, the C is used to derive the COL. Then, `availableFlagLXCcol` is set equal to 1. Note that the reference picture may be a `collocated_ref_idx` signaled in the slice header.

The pairwise candidate derivation unit derives a pairwise candidate `avgK` from the average of the two merge candidates (`p0Cand` and `p1Cand`) stored in `mergeCandList` and stores the pairwise candidate `avgK` in `mergeCandList[ ]`.

$$\begin{aligned} mvLXavgK[0] &= (mvLXp0Cand[0] + mvLXp1Cand[0])/2 \\ mvLXavgK[1] &= (mvLXp0Cand[1] + mvLXp1Cand[1])/2 \end{aligned}$$

The merge candidate derivation unit **30361** derives zero merge candidates `Z0`, . . . , `ZM` in which `refIdxLX` is 0. . . `M` and in which an X component and a Y component of `mvLX` are both 0, and store the zero merge candidates in the merge candidate list.

The storage in `mergeCandList[ ]` is in the order of, for example, spatial merge candidates (`A1`, `B1`, `B0`, `A0`, and `B2`), the temporal merge candidate `Col`, the pairwise merge candidate `avgK`, and the zero merge candidate `ZK`. Note that a reference block that is not available (intra prediction block, or the like) is not stored in the merge candidate list.

$$\begin{aligned} i &= 0 \\ \text{if}(\text{availableFlagA1}) \\ \text{mergeCandList}[i++] &= \text{A1} \end{aligned}$$

20

-continued

```

if(availableFlagB1)
mergeCandList[i++] = B1
if(availableFlagB0)
mergeCandList[i++] = B0
if(availableFlagA0)
mergeCandList[i++] = A0
if(availableFlagB2)
mergeCandList[i++] = B2
if(availableFlagCol)
mergeCandList[i++] = Col
if(availableFlagAvgK)
mergeCandList[i++] = avgK
if(i < MaxNumMergeCand)
mergeCandList[i++] = ZK

```

The merge candidate selection unit **30362** selects a merge candidate `N` indicated by `merge_idx` from the merge candidates included in the merge candidate list, in accordance with the equation below.

$$N = \text{mergeCandList}[\text{merge_idx}]$$

Here, `N` is a label indicating a merge candidate, and takes `A1`, `B1`, `B0`, `A0`, `B2`, `Col`, `avgK`, `ZK`, and the like. The motion information of the merge candidate indicated by the label `N` is indicated by (`mvLXN[0]`, `mvLXN[1]`), `predFlagLXN`, and `refIdxLXN`.

(`mvLXN[0]`, `mvLXN[1]`), `predFlagLXN`, and `refIdxLXN` selected are selected as inter prediction parameters for the target block. The merge candidate selection unit **30362** stores the inter prediction parameter for the selected merge candidate in the prediction parameter memory **307** and outputs the inter prediction parameter to the inter prediction image generation unit **309**.

DMVR

Now, a Decoder side Motion Vector Refinement (DMVR) processing performed by the DMVR unit **30537** will be described. In a case that the `merge_flag` is 1 or the skip flag `skip_flag` is 1 for the target CU, the DMVR unit **30537** refines `mvLX` of the target CU derived by the merge prediction unit **30374** by using the reference image. Specifically, in a case that the prediction parameter derived by the merge prediction unit **30374** indicates bi-prediction, the motion vector is refined by using the prediction image derived from the motion vector corresponding to two reference pictures. `mvLX` refined is supplied to the inter prediction image generation unit **309**.

In the derivation of the flag `dmvrFlag` specifying whether to perform DMVR processing, one of multiple conditions for setting `dmvrFlag` to 1 is that the value of `RefPicIsScaled[0][refIdxL0]` described above is 0 and the value of `RefPicIsScaled[1][refIdxL1]` described above is 0. In a case that the value of `dmvrFlag` is set equal to 1, the DMVR unit **30537** performs DMVR processing.

In the derivation of the flag `dmvrFlag` specifying whether to perform DMVR processing, one of the multiple conditions for setting `dmvrFlag` to 1 is that `luma_weight_10_flag[i]` is 0, `luma_weight_10_flag[i]` corresponding to a flag indicating whether coefficient information for a weighted prediction of an L0 prediction of luminance described below is present, and that the value of `luma_weight_11_flag[i]` is 0, `luma_weight_11_flag[i]` corresponding to a flag indicating whether coefficient information for a weighted prediction of

21

an L1 prediction of luminance described below is present. In a case that the value of dmvrFlag is set equal to 1, the DMVR unit **30537** performs DMVR processing.

Note that, in the derivation of the flag dmvrFlag specifying whether to perform DMVR processing, one of the multiple conditions for setting dmvrFlag to 1 may be that luma_weight_10_flag[i] is 0 and the value of luma_weight_11_flag[i] is 0 and that chroma_weight_10_flag[i] is 0, chroma_weight_10_flag[i] corresponding to a flag indicating whether coefficient information for a weighted prediction of an L0 prediction of chrominance described below is present, and that the value of chroma_weight_11_flag[i] is 0, chroma_weight_11_flag[i] corresponding to a flag indicating whether coefficient information for a weighted prediction of an L1 prediction of chrominance described below is present. In a case that the value of dmvrFlag is set equal to 1, the DMVR unit **30537** performs DMVR processing.

(Prof)

In a case that the value of RefPicIsScaled[0][refIdxLX] is 1 or the value of RefPicIsScaled[1][refIdxLX] is 1, the value of cbProfFlagLX is set equal to FALSE. Here, cbProfFlagLX is a flag that specifies whether to perform Prediction refinement (PROF) of the affine prediction.

AMVP Prediction

FIG. **12** is a schematic diagram illustrating a configuration of the AMVP prediction parameter derivation unit **3032** according to the present embodiment. The AMVP prediction parameter derivation unit **3032** includes a vector candidate derivation unit **3033** and a vector candidate selection unit **3034**. The vector candidate derivation unit **3033** derives a prediction vector candidate from the motion vector for the decoded adjacent block stored in the prediction parameter memory **307** based on refIdxLX, and stores the result in a prediction vector candidate list mvplstLX[].

The vector candidate selection unit **3034** selects a motion vector mvplstLX[mvp_LX_idx] indicated by mvp_LX_idx, among the prediction vector candidates of the prediction vector candidate list mvplstLX[], as mvplX. The vector candidate selection unit **3034** outputs mvplX selected to the MV addition unit **3038**.

MV Addition Unit

The MV addition unit **3038** adds mvplX input from the AMVP prediction parameter derivation unit **3032** and mvdLX decoded, to calculate mvLX. The addition unit **3038** outputs mvLX calculated to the inter prediction image generation unit **309** and the prediction parameter memory **307**.

$$mvLX[0] = mvplX[0] + mvdLX[0]$$

$$mvLX[1] = mvplX[1] + mvdLX[1]$$

Detailed Classification of Subblock Merge

An overview of the types of prediction processing associated with subblock merge will be provided. As described above, the types of prediction processing associated with subblock merge is roughly divided into a merge prediction and an AMVP prediction.

The merge prediction is further classified as follows.

Regular merge prediction (block-based merge prediction)

Subblock merge prediction

22

The subblock merge predictions is further classified as follows.

Subblock prediction (ATMVP)

Affine prediction

Inferred affine prediction

Constructed affine prediction

On the other hand, the AMVP prediction is classified as follows.

AMVP (translation)

MVD affine prediction

The MVD affine prediction is further classified as follows.

4 parameter MVD affine prediction

6 parameter MVD affine prediction

Note that the MVD affine prediction refers to an affine prediction in which a difference vector is decoded for use.

In the subblock prediction, as in the temporal merge derivation processing, the availability availableFlagSbCol of the collocated subblock COL of the target subblock is determined, and in a case that the collocated subblock is available, the prediction parameter is derived. At least in a case that SliceTemporalMvpEnabledFlag described above is 0, availableFlagSbCol is set equal to 0.

The Merge with Motion Vector Difference (MMVD) prediction may be classified as a merge prediction or as an AMVP prediction. In the former case, mmvd_flag and MMVD-related syntax elements are decoded in a case of merge_flag=1. In the latter case, mmvd_flag and the MMVD-related syntax elements are decoded in a case of merge_flag=0.

The loop filter **305** is a filter provided in the coding loop, and is a filter that removes block distortion and ringing distortion and improves image quality. The loop filter **305** applies a filter such as a deblocking filter, a Sample Adaptive Offset (SAO), and an Adaptive Loop Filter (ALF) on a decoded image of a CU generated by the addition unit **312**.

The reference picture memory **306** stores a decoded image of the CU in a predefined position for each target picture and target CU.

The prediction parameter memory **307** stores the prediction parameter in a predefined position for each CTU or CU. Specifically, the prediction parameter memory **307** stores the parameter decoded by the parameter decoder **302**, the parameter derived by the prediction parameter derivation unit **320**, and the like.

Parameters derived by the prediction parameter derivation unit **320** are input to the prediction image generation unit **308**. In addition, the prediction image generation unit **308** reads a reference picture from the reference picture memory **306**. The prediction image generation unit **308** generates a prediction image of a block or a subblock by using the parameters and the reference picture (reference picture block) in the prediction mode indicated by predMode. Here, the reference picture block refers to a set of pixels (referred to as a block because they are normally rectangular) on a reference picture and is a region that is referenced for generating a prediction image.

Inter Prediction Image Generation Unit **309**

In a case that predMode indicates the inter prediction mode, the inter prediction image generation unit **309** generates a prediction image of a block or a subblock by inter prediction by using the inter prediction parameters input from the inter prediction parameter derivation unit **303** and the read reference picture.

FIG. **14** is a schematic diagram illustrating a configuration of the inter prediction image generation unit **309** included in the prediction image generation unit **308** according to the present embodiment. The inter prediction image generation unit **309** includes a motion compensation unit (prediction image generation unit) **3091** and a combining unit **3095**. The combining unit **3095** includes an IntraInter combining unit

23

30951, a Triangle combining unit **30952**, a BDOF unit **30954**, and a weighted prediction unit **3094**.

Motion Compensation

The motion compensation unit **3091** (interpolation image generation unit **3091**) generates an interpolation image (motion compensation image) by reading a reference block from the reference picture memory **306** based on the inter prediction parameters (predFlagLX, refIdxLX, mvLX) input from the inter prediction parameter derivation unit **303**. The reference block is a block located on the reference picture RefPicLX designated by refIdxLX, at a position shifted by mvLX from the position of the target block. Here, in a case that mvLX does not have an integer precision, an interpolation image is generated by using a filter referred to as a motion compensation filter and configured to generate pixels at fractional positions.

The motion compensation unit **3091** first derives an integer position (xInt, yInt) and a phase (xFrac, yFrac) corresponding to in-prediction block coordinates (x, y) by the following equation.

$$\begin{aligned} xInt &= xPb + (mvLX[0] \gg (\log_2(MVPREC))) + x \\ xFrac &= mvLX[0] \& (MVPREC - 1) \\ yInt &= yPb + (mvLX[1] \gg (\log_2(MVPREC))) + y \\ yFrac &= mvLX[1] \& (MVPREC - 1) \end{aligned}$$

Here, (xPb, yPb) indicates the upper left coordinates of a block with a bW*bH size, that is, x=0 . . . bW-1, y=0 . . . bH-1, and MVPREC indicates the precision of the motion vector mvLX (1/MVPREC pixel precision). For example, MVPREC=16.

The motion compensation unit **3091** derives a temporary image temp[][] by performing horizontal interpolation processing on a reference picture refImg using an interpolation filter. In the equation below, Σ is the sum related to k of k=0, . . . , NTAP-1, shift1 is a normalization parameter for adjusting a value range, and offset1=1<<(shift1-1).

$$\text{temp}[x][y] = (\Sigma \text{mcFilter}[xFrac][k] * \text{refImg}[xInt + k - \text{NTAP}/2 + 1][yInt + \text{offset1}] \gg \text{shift1})$$

Subsequently, the motion compensation unit **3091** derives an interpolation image Pred[][] by performing vertical interpolation processing on the temporary image temp[][]. In the equation below, Σ is the sum related to k of k=0, . . . , NTAP-1, shift2 is a normalization parameter for adjusting the value range, and offset2=1<<(shift2-1).

$$\text{Pred}[x][y] = (\Sigma \text{mcFilter}[yFrac][k] * \text{temp}[x][y + k - \text{NTAP}/2 + 1] + \text{offset2}) \gg \text{shift2}$$

Note that for bi-prediction, Pred[][] described above is derived for each of the L0 list and the L1 list (referred to as interpolation images PredL0[][] and PredL1[][]), and an interpolation image Pred[][] is generated from PredL0[][] and PredL1[][].

Note that the motion compensation unit **3091** has a function of scaling an interpolation images in accordance with the scaling ratio in the horizontal direction RefPicScale[i][j][0] of the reference picture derived by the scale parameter derivation unit **30378**, and the scaling ratio in the vertical direction RefPicScale[i][j][1] of the reference picture.

24

The combining unit **3095** includes the IntraInter combining unit **30951**, the Triangle combining unit **30952**, the weighted prediction unit **3094**, and the BDOF unit **30954**. Interpolation Filter Processing

Interpolation filter processing, that is performed by the prediction image generation unit **308**, in a case that the resampling described above is applied to change the size of the reference picture within a single sequence, will be described below. Note that this processing may be performed by, for example, the motion compensation unit **3091**.

The prediction image generation unit **308** performs the interpolation filter processing described below in a case that the value of RefPicIsScaled[i][j] received from the inter prediction parameter derivation unit **303** indicates that the reference picture is scaled.

In correcting the size of the reference picture for luminance, the prediction image generation unit **308** derives the value of a horizontal luminance interpolation filter coefficient fLH[p] and the value of a vertical luminance interpolation filter coefficient fLV[p]. Here, the value p corresponds to every 1/16 position in the horizontal or vertical direction of a luminance sample. The horizontal luminance interpolation filter coefficient fLH[p] of a 1/16 pixel position p of xFracL or yFracL is derived as follows.

In the affine prediction, in a case that the block size is equal to 4 in both the horizontal direction and the vertical direction, any of the coefficients in FIG. **18** is used as the luminance interpolation filter coefficient fLH[p].

Otherwise, in a case that scalingRatio[0] is greater than 28672, any of the coefficients in FIG. **16** is used as the luminance interpolation filter coefficient fLH[p].

Otherwise, in a case that the scalingRatio[0] is greater than 20480, any of the coefficients in FIG. **17** is used as the luminance interpolation filter coefficient fLH[p].

Otherwise, the luminance interpolation filter coefficient fLH[p] is indicated in FIG. **15** in accordance with hpellfIdx.

The vertical luminance interpolation filter coefficients fLV[p] of each 1/16 terminal sample position p equal to yFracL is derived as follows.

In the affine prediction, in a case that the block size is equal to 4 in both the horizontal direction and the vertical direction, any of the coefficients in FIG. **18** is used as the luminance interpolation filter coefficient fLV[p].

Otherwise, in a case that the scalingRatio[1] is greater than 28672, any of the coefficients in FIG. **16** is used as the luminance interpolation filter coefficient fLV[p].

Otherwise, in a case that the scalingRatio[1] is greater than 20480, any of the coefficients in FIG. **17** is used as the luminance interpolation filter coefficient fLV[p].

Otherwise, any of the coefficients in FIG. **15** is used as the luminance interpolation filter coefficient fLV[p] in accordance with hpellfIdx.

With the integer pixel position of luminance in a case of i=0, . . . , 7 being represented as (xInti, yInti), the pixel position of a fraction of luminance being represented as (xFracL, yFracL), a reference pixel array of luminance being represented as refPicLXL, and the index of a 1/2 pixel interpolation filter being represented as hpellfIdx, the pixel value predSampleLXL of the interpolation image of luminance is derived as follows.

In a case that both xFracL and yFracL are equal to 0 and that both scalingRatio[0] and scalingRatio[1] are smaller than 20481, then the value of predSampleLXL is derived as follows. In the description below, 2 denotes a sum related to i=0, . . . , 7.

25

$$\text{predSampleLXL} = \text{refPicLXL}[\text{xInt3}][\text{yInt3}] \ll \text{shift3}$$

Otherwise, in a case that yFracL is 0 and that $\text{scalingRatio}[1]$ is smaller than 20481, the value of predSampleLXL is derived as follows.

$$\text{predSampleLXL} = (\Sigma \text{fLH}[\text{xFracL}][i] * \text{refPicLXL}[\text{xInt1}][\text{yInt3}]) \gg \text{shift1}$$

Otherwise, in a case that xFracL is 0 and that $\text{scalingRatio}[1]$ is smaller than 20481, the value of predSampleLXL is derived as follows.

$$\text{predSampleLXL} = (\Sigma \text{fLV}[\text{yFracL}][i] * \text{refPicLXL}[\text{xInt3}][\text{yInt1}]) \gg \text{shift1}$$

Otherwise, the value of predSampleLXL is derived as follows.

A pixel array $\text{temp}[n]$ with $n=0, \dots, 7$ is derived as follows.

$$\text{temp}[n] = (\Sigma \text{fLH}[\text{xFracL}][i] * \text{refPicLXL}[\text{xInt1}][\text{yIntn}]) \gg \text{shift1}$$

The pixel value predSampleLXL of the interpolation image of the luminance is derived as follows.

$$\text{predSampleLXL} = (\Sigma \text{fLV}[\text{yFracL}][i] * \text{temp}[i]) \gg \text{shift2}$$

In correcting the size of the reference picture for chrominance, the prediction image generation unit 308 derives the value of a horizontal chrominance interpolation filter coefficient $\text{fCH}[p]$ and the value of a vertical chrominance interpolation filter coefficient $\text{fCV}[p]$. Here, the value p corresponds to every $1/32$ position in the horizontal or vertical direction of a chrominance sample. The horizontal chrominance interpolation filter coefficient $\text{fCH}[p]$ of a $1/32$ pixel position p of xFracC is derived as follows.

In a case that $\text{scalingRatio}[0]$ is greater than 28672, any of the coefficients in FIG. 21 is used as the chrominance interpolation filter coefficient $\text{fCH}[p]$.

Otherwise, in a case that $\text{scalingRatio}[0]$ is greater than 20480, any of the coefficients in FIG. 20 is used as the chrominance interpolation filter coefficient $\text{fCH}[p]$.

Otherwise, any of the coefficients in FIG. 19 is used as the chrominance interpolation filter coefficient $\text{fCH}[p]$.

The vertical chrominance interpolation filter coefficient $\text{fCV}[p]$ of each $1/32$ fraction pixel position p equal to yFracC is derived as follows.

In a case that $\text{scalingRatio}[1]$ is greater than 28672, any of the coefficients in FIG. 21 is used as the chrominance interpolation filter coefficient $\text{fCV}[p]$.

Otherwise, in a case that $\text{scalingRatio}[1]$ is greater than 20480, any of the coefficients in FIG. 20 is used as the chrominance interpolation filter coefficient $\text{fCV}[p]$.

Otherwise, any of the coefficients in FIG. 19 is used as the chroma interpolation filter coefficient $\text{fCV}[p]$.

With the integer pixel position of chrominance in a case of $i=0, \dots, 3$ being represented as $(\text{xInti}, \text{yInti})$, the pixel position of a fraction of chrominance being represented as $(\text{xFracL}, \text{yFracL})$, and the reference pixel array of chrominance being represented as refPicLXL , the pixel value predSampleLXL of the interpolation image of chrominance is derived as follows.

26

The pixel value predSampleLXC of the interpolation image of chrominance is derived as follows.

In a case that both xFracC and yFracC are 0 and that both $\text{scalingRatio}[0]$ and $\text{scalingRatio}[1]$ are smaller than 20481, the value of predSampleLXC is derived as follows. Hereinafter, Σ denotes the sum of $i=0, \dots, 3$.

$$\text{predSampleLXC} = \text{refPicLXC}[\text{xInt1}][\text{yInt1}] \ll \text{shift3}$$

Otherwise, in a case that yFracC is 0 and that $\text{scalingRatio}[1]$ is smaller than 20481, the value of predSampleLXC is derived as follows.

$$\text{predSampleLXC} = (\Sigma \text{fCH}[\text{xFracC}][i] * \text{refPicLXC}[\text{xInt1}][\text{yInt1}]) \gg \text{shift1}$$

Otherwise, in a case that xFracC is 0 and that $\text{scalingRatio}[0]$ is smaller than 20481, the value of predSampleLXC is derived as follows.

$$\text{predSampleLXC} = (\Sigma \text{fCV}[\text{yFracC}][i] * \text{refPicLXC}[\text{xInt1}][\text{yInti}]) \gg \text{shift1}$$

Otherwise, the value of predSampleLXC is derived as follows, and the pixel array $\text{temp}[n]$ with $n=0, \dots, 3$ is derived as follows.

$$\text{temp}[n] = (\Sigma \text{fCH}[\text{xFracC}][i] * \text{refPicLXC}[\text{xInti}][\text{yIntn}]) \gg \text{shift1}$$

The pixel value predSampleLXC of the interpolation image of chrominance is derived as follows.

$$\begin{aligned} \text{predSampleLXC} = & (\text{fCV}[\text{yFracC}][0] * \text{temp}[0] + \text{fCV}[\text{yFracC}][1] * \\ & \text{temp}[1] + \text{fCV}[\text{yFracC}][2] * \text{temp}[2] + \text{fCV}[\text{yFracC}][3] * \\ & \text{temp}[3]) \gg \text{shift2} \end{aligned}$$

IntraInter Combining Processing

The IntraInter combining unit 30951 generates a prediction image through the weighted sum of an inter prediction image and an intra prediction image.

Triangle Combining Processing

The Triangle combining unit 30952 generates a prediction image using the Triangle prediction described above.

BDOF Prediction

Now, the details of a Bi-Directional Optical Flow (BDOF) processing, BDOF) prediction performed by the BDOF unit 30954 will be described. In a bi-prediction mode, the BDOF unit 30954 generates a prediction image with reference to two prediction images (first prediction image and second prediction image) and a gradient correction term.

FIG. 25 is a flowchart illustrating a flow of processing for deriving a prediction image.

In a case that the inter prediction parameter decoder 303 determines an L0 uni-prediction (in S101, inter_pred_idc is 0), the motion compensation unit 3091 generates $\text{PredL0}[x][y]$ (S102). In a case that the inter prediction parameter decoder 303 determines an L1 uni-prediction (in S101, inter_pred_idc is 1), the motion compensation unit 3091 generates $\text{PredL1}[x][y]$ (S103). On the other hand, in a case that the inter prediction parameter decoder 303 is determined to be in a bi-prediction mode (in S101, inter_pred_idc is 2), processing in S104 described below is subsequently performed. In S104, the combining unit 3095 references

bdoFAvailableFlag indicating whether to perform BDOF processing to determine whether the BDOF processing is necessary. In a case that bdoFAvailableFlag indicates TRUE, the BDOF unit **30954** performs the BDOF processing to generate a bi-prediction image (S106), and in response to bdoFAvailableFlag indicating FALSE, the combining unit **3095** generates a prediction image by normal bi-directional prediction image generation (S105).

In the derivation of the flag bdoFAvailableFlag specifying whether to perform BDOF processing, one of the multiple conditions for setting bdoFAvailableFlag to 1 is that the value of RefPicLsScaled[0][refIdxL0] described above is 0 and that the value of RefPicLsScaled[1][refIdxL1] described above is 0.

In the derivation of the flag bdoFAvailableFlag specifying whether to perform the BDOF processing, one of the multiple conditions for setting bdoFAvailableFlag to 1 is that luma_weight_10_flag[i] is 0, luma_weight_10_flag[i] corresponding to a flag indicating whether coefficient information for a weighted prediction of an L0 prediction of luminance described below is present, and that the value of luma_weight_11_flag[i] is 0, luma_weight_11_flag[i] corresponding to a flag indicating whether coefficient information for a weighted prediction of an L1 prediction of luminance described below is present.

Note that in the derivation of the flag bdoFAvailableFlag specifying whether to perform the BDOF processing, one of the multiple conditions for setting bdoFAvailableFlag to 1 may be that in the inter prediction parameter decoder **303**, luma_weight_10_flag[i] is 0 and the value of luma_weight_11_flag[i] is 0 and that chroma_weight_10_flag[i] is 0, chroma_weight_10_flag[i] corresponding to a flag indicating whether coefficient information for a weighted prediction of an L0 prediction of chrominance described below is present, and that the value of chroma_weight_11_flag[i] is 0, chroma_weight_11_flag[i] corresponding to a flag indicating whether coefficient information for a weighted prediction of an L1 prediction of luminance described below is present.

By using FIG. 26, the specific contents of processing performed by the BDOF unit **30954** will be described. The BDOF unit **30954** includes an L0, L1 prediction image generation unit **309541**, a gradient image generation unit **309542**, a correlation parameter calculation unit **309543**, a motion compensation correction value derivation unit **309544**, and a bi-prediction image generation unit **309545**. The BDOF unit **30954** generates a prediction image from the interpolation image received from the motion compensation unit **3091** and the inter prediction parameter received from the inter prediction parameter decoder **303**, and outputs the prediction image to the addition unit **312**. Note that processing for deriving a motion compensation correction value bdoFOffset (motion compensation corrected image) from a gradient image and correcting and deriving PredL0 and PredL1 is referred to as bi-directional optical flow sample prediction process.

The L0, L1 prediction image generation unit **309541** generates interpolation images predSamplesL0 and predSamplesL1 for L0 and L1 predictions, the interpolation images being used for the BDOF processing.

Note that the variable shift1 is set equal to 6, the variable shift2 is set equal to 4, the variable shift3 is set equal to 1, the variable shift4 is set equal to Max (3, 15-BitDepth), the variable offset4 is set to 1<<(shift4-1), and the variable mvRefineThres is set to 1<<4. predSamplesL0 is an interpolation image of the L0 prediction, predSamplesL1 is an interpolation image of the L1 prediction.

In a case that bdoFAvailableFlag is FALSE, then in the bi-prediction image generation unit **309545**, the pixel value of the prediction image is derived in units of 4×4 pixel blocks as follows.

```
pbSamples[x][y] = Clip3(0, (1 << BitDepth) - 1,
    (predSamplesL0[x + 1][y + 1] + offset4 +
    predSamplesL1[x + 1][y + 1]) >>
```

shift4)

Otherwise (bdoFAvailableFlag is equal to TRUE), the pixel value of the current prediction image is derived as follows.

The position (hx, vy) of the corresponding pixel position (x, y) in the prediction image array is derived as follows.

```
hx = Clip3(1, W, x)
vy = Clip3(1, H, y)
```

Here, W is the number of pixels in the horizontal direction of a processing block, and H is the number of pixels in the vertical direction of the processing block.

The gradient image generation unit **309542** generates a gradient image. Specifically, the variables gradientHL0[x][y], gradientVL0[x][y], gradientHL1[x][y], gradientVL1[x][y] of the gradient image are derived as follows. gradientHL0 and gradientHL1 represent gradient images in the horizontal direction, and gradientVL0 and gradientVL1 represent gradient images in the vertical direction.

```
gradientHL0[x][y] = (predSamplesL0[hx + 1][vy] >> shift1)
    - (predSampleL0[hx - 1][vy]) >> shift1)
gradientVL0[x][y] = (predSampleL0[hx][vy + 1] >> shift1)
    - (predSampleL0[hx][vy - 1]) >> shift1)
gradientHL1[x][y] = (predSamplesL1[hx + 1][vy] >> shift1)
    - (predSampleL1[hx - 1][vy]) >> shift1)
gradientVL1[x][y] = (predSampleL1[hx][vy + 1] >> shift1)
    - (predSampleL1[hx][vy - 1]) >> shift1)
The variables diff[x][y], tempH[x][y], and tempV[x][y] are derived.
diff[x][y] = (predSamplesL0[hx][vy] >> shift2) -
    (predSamplesL1[hx][vy] >> shift2)
tempH[x][y] = (gradientHL0[x][y] + gradientHL1[x][y]) >> shift3
tempV[x][y] = (gradientVL0[x][y] + gradientVL1[x][y]) >> shift3
```

then, the correlation parameter calculation unit **309543** derives gradient product-sum variables sGx2, sGy2, sGxGy, sGxDI, and sGyDI for each block of 4×4 pixels as follows.

```
sGx2 = ΣΣabs(tempH[xSb + i][ySb + j])
sGy2 = ΣΣabs(tempV[xSb + i][ySb + j])
sGxGy = ΣΣ(Sign(tempV[xSb + i][ySb + j]) * tempH[xSb + i][ySb + j])
sGxDI = ΣΣ(-Sign(tempH[xSb + i][ySb + j]) * diff[xSb + i][ySb + j])
sGyDI = ΣΣ(-Sign(tempV[xSb + i][ySb + j]) * diff[xSb + i][ySb + j])
```

Here, ΣΣ means that, with i=-1, . . . , 4 and j=-1, . . . , 4, the total value of a block of 6×6 pixels including 4×4 pixels and one pixel around the 4×4 pixels is determined, abs(x) is a function that returns the absolute value of x, and Sign(x) is a function that returns 1 in a case that the value of x is greater than 0 and that returns 0 in a case that the value of x is 0 and that returns -1 in a case that the value of x is smaller than 0.

Then, the motion compensation correction value derivation unit **309544** derives a corrected weight vector (u, v) in units of 4×4 pixels by using the derived gradient product-sums sGx2, sGy2, sGxGy, sGxDI, and sGyDI.

```

u = sGx2 > 0?Clip3(-mvRefineThres + 1, mvRefineThres - 1,
(sGxdI << 2) >> Floor(Log2(sGx2))):0
v = sGy2 > 0?Clip3(-mvRefineThres + 1, mvRefineThres - 1,
((sGydI << 2) - ((vx * sGxGy) >> 1)) >> Floor(Log2(sGy2))):0

```

The motion compensation correction value derivation unit **309544** derives bdofoffset by using the corrected weight vector (u, v) in units of 4×4 pixels and the gradient images gradientHL0, gradientVL0, gradientHL1, and gradientVL1.

```

bdofoffset = u * (gradientHL0[x+1][y+ 1] - gradientHL1[x + 1][y + 1])
+ v * (gradientVL0[x + 1][y + 1] - gradientVL1[x + 1][y + 1])

```

The bi-prediction image generation unit **309545** derives, by using the above-described parameters, the pixel value Pred of the prediction image of the 4×4 pixels in accordance with the following equation.

```

pbSamples[x][y] = Clip3(0, (1 << BitDepth) - 1,
(predSamplesL0[x + 1][y + 1] + offset4 +
predSamplesL1[x + 1][y + 1] + bdofoffset) >> shift4)

```

Weighted Prediction

The weighted prediction unit **3094** generates a prediction image pbSamples of the block from an interpolation image PredLX. First, a syntax associated with the weighted prediction unit **3094** will be described.

FIG. 27(a) illustrates an example of a syntax of sps_weighted_pred_flag and sps_weighted_bipred_flag for the Sequence Parameter Set (SPS).

sps_weighted_pred_flag being equal to 1 indicates that the weighted prediction may be applied to P slices that reference the SPS. sps_weighted_pred_flag being equal to 0 indicates that the weighted prediction is not applied to the P slices that reference the SPS. sps_weighted_bipred_flag being equal to 1 indicates that the weighted prediction may be applied to B slices that reference the SPS. sps_weighted_bipred_flag being equal to 0 indicates that the weighted prediction is not applied to the B slices that reference the SPS.

FIG. 27(b) illustrates an example of a syntax of pps_weighted_pred_flag and pps_weighted_bipred_flag for the Picture Parameter Set (PPS).

pps_weighted_pred_flag being equal to 0 indicates that the weighted prediction is not applied to P slices that reference the PPS. pps_weighted_pred_flag being equal to 1 indicates that the weighted prediction is applied to the P slices that reference the PPS. In a case that sps_weighted_pred_flag is equal to 0, the weighted prediction unit **3094** sets the value of pps_weighted_pred_flag to 0. In a case that pps_weighted_pred_flag is not present, then the value is set equal to 0.

pps_weighted_bipred_flag being equal to 0 indicates that the weighted prediction is not applied to B slices that reference the PPS. pps_weighted_bipred_flag being equal to 1 indicates that the weighted prediction is applied to the B slices that reference the PPS. In a case that sps_weighted_bipred_flag is equal to 0, the weighted prediction unit **3094** sets the value of pps_weighted_bipred_flag to 0. In a case that pps_weighted_bipred_flag is not present, then the value is set equal to 0.

FIG. 28 illustrates the Weighted prediction parameters syntax () of NPL 1, which is invoked by a slice header.

Here, luma_log2_weight_denom is a base 2 logarithm for the denominators of all luminance weight coefficients. The value of luma_log2_weight_denom need to be in a range from 0 to 7. delta_chroma_log2_weight_denom is a difference in base 2 logarithm for the denominators of all chrominance weight coefficients. In a case that delta_chroma_log2_weight_denom is absent, delta_chroma_log2_weight_denom is inferred to be equal to 0. The variable ChromaLog2WeightDenom is derived to be equal to luma_log2_weight_denom + delta_chroma_log2_weight_denom, and the value needs to be in a range from 0 to 7.

luma_weight_10_flag[i] being 1 indicates that a weight coefficient of the luminance component for the L0 prediction is present. luma_weight_10_flag[i] being 0 indicates that a weight coefficient of the luminance component for the L0 prediction is absent. In a case that the luma_weight_10_flag[i] is absent, the weighted prediction unit **3094** infers that luma_weight_10_flag[i] is equal to 0. chroma_weight_10_flag[i] being 1 indicates that a weight coefficient of the chrominance prediction value for the L0 prediction is present. chroma_weight_10_flag[i] being 0 indicates that a weight coefficient of the chrominance prediction value for the L0 prediction is absent. In a case that chroma_weight_10_flag[i] is absent, then the weighted prediction unit **3094** infers that chroma_weight_10_flag[i] is equal to 0.

delta_luma_weight_10[i] is a difference in weight coefficient applied to the luminance prediction value for the L0 prediction using RefPicList[0][i]. The variable LumaWeightL0[i] is derived to be equal to (1<<luma_log2_weight_denom)+delta_luma_weight_10[i]. In a case that luma_weight_10_flag[i] is equal to 1, the value of delta_luma_weight_10[i] needs to be in a range from -128 to 127. In a case that luma_weight_10_flag[i] is equal to 0, then the weighted prediction unit **3094** infers that LumaWeightL0[i] is equal to the value of 2 to the luma_log2_weight_denom-th power (2^{luma_log2_weight_denom}). luma_offset_10[i] is an addition offset applied to the luminance prediction value for the L0 prediction using RefPicList[0][i]. The value of luma_offset_10[i] needs to be in a range from -128 to 127. In a case that luma_weight_10_flag[i] is equal to 0, then the weighted prediction unit **3094** infers that luma_offset_10[i] is equal to 0.

delta_chroma_weight_10[i][j] is a difference in weight coefficient applied to the chroma prediction value for the L0 prediction using RefPicList0[i] with j of 0 for Ch and j of 1 for Cr. The variable ChromaWeightL0[i][j] is derived to be equal to (1<<ChromaLog2WeightDenom)+delta_chroma_weight_10[i][j]. In a case that chroma_weight_10_flag[i] is equal to 1, the value of delta_chroma_weight_10[i][j] needs to be in a range from -128 to 127. In a case that chroma_weight_10_flag[i] is 0, then the weighted prediction unit **3094** infers that ChromaWeightL0[i][j] is equal to the value of 2 to the ChromaLog2WeightDenom-th power (2^{ChromaLog2WeightDenom}). delta_chroma_offset_10[i][j] is a difference in addition offset applied to the chroma prediction value for the L0 prediction using RefPicList0[i] with j of 0 for Cb and j of 1 for Cr. The variable ChromaOffsetL0[i][j] is derived as follows.

31

```
ChromaOffsetL0[i][j] = Clip3(-128, 127,
(128 + delta_chroma_offset_10[i][j] -
((128 * ChromaWeightL0[i][j]) >> ChromaLog2WeightDenom)))
```

The value of delta_chroma_offset_10[i][j] needs to be in a range from -4*128 to 4*127. In a case that chroma_weight_10_flag[i] is equal to 0, the weighted prediction unit **3094** infers that ChromaOffsetL0[i][j] is equal to 0.

Note that luma_weight_11_flag[i], chroma_weight_11_flag[i], delta_luma_weight_11[i], luma_offset_11[i], delta_chroma_weight_11[i][j], and delta_chroma_offset_11[i][j] are respectively interpreted as luma_weight_10_flag[i], chroma_weight_10_flag[i], delta_luma_weight_10[i], luma_offset_10[i], delta_chroma_weight_10[i][j], and delta_chroma_offset_10[i][j], and that 10, L0, list0, and List0 are respectively interpreted as 11, 11, list1, and List1.

The processing of the inter prediction parameter derivation unit **303** will be described below.

In NPL 1, the variable weightedPredFlag indicating whether to perform the weighted prediction processing is derived as follows.

In a case that slice_type is equal to P, weightedPredFlag is set equal to pps_weighted_pred_flag defined by the PPS.

Otherwise, in a case that slice_type is equal to B, weightedPredFlag is set equal to pps_weighted_bipred_flag && (!dmvrFlag) defined by the PPS.

Hereinafter, bcw_idx is the weight index of a bi-prediction with weights in units of CU. In a case that bcw_idx is not signaled, then bcw_idx=0 is set. For bcw_idx, the bcwIdxN of a neighboring block is set in the merge prediction mode, and the bow_idx of the target block is set in the AMVP prediction mode.

In a case that the value of the variable weightedPredFlag is equal to 0 or that the value of the variable bcwIdx is 0, then the normal prediction image processing is performed, and the prediction image pbSamples is derived as follows.

In a case that one of prediction list utilization flags (predFlagL0 or predFlagL1) is 1 (uni-prediction) (no weighted prediction is used), processing in accordance with the equation below is performed in which PredLX (LX is L0 or L1) is adapted to the number of pixel bits bitDepth.

```
pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1, (PredLX[x][y] +
offset1) >> shift1)
```

Here, shift1 = 14 - bitDepth, offset1 = 1 << (shift1 - 1) are established. PredLX is an interpolation image of the L0 or L1 prediction.

In a case that both of prediction list utilization flags (predFlagL0 and predFlagL1) are 1 (bi-prediction PRED_BI) and that no weight prediction is used, processing in accordance with the equation below is executed in which PredL0 and PredL1 are averaged and adapted to the number of pixel bits.

```
pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1, (PredL0[x][y] +
PredL1[x][y] + offset2) >> shift2)
```

Here, shift2 = 15 - bitDepth, offset2 = 1 << (shift2 - 1) are established.

In a case that the value of the variable weightedPredFlag is equal to 1 and that the value of the variable bcwIdx is equal to 0, then the weighted prediction processing is performed, and the prediction image pbSamples is derived as follows.

32

The variable shift1 is set equal to Max(2, 14-bitDepth). The variables log2Wd, o0, o1, w0, and w1 are derived as follows.

In a case that cIdx is 0, corresponding to luminance, then the following applies.

```
log2Wd = luma_log2_weight_denom + shift1
w0 = LumaWeightL0[refIdxL0]
w1 = LumaWeightL1[refIdxL1]
o0 = luma_offset_10[refIdxL0] << (bitDepth - 8)
o1 = luma_offset_11[refIdxL1] << (bitDepth - 8)
```

Otherwise (cIdx is a chrominance not equal to 0), the following applies.

```
log2Wd = ChromaLog2WeightDenom + shift1
w0 = ChromaWeightL0[refIdxL0][cIdx - 1]
w1 = ChromaWeightL1[refIdxL1][cIdx - 1]
o0 = ChromaOffsetL0[refIdxL0][cIdx - 1] << (bitDepth - 8)
o1 = ChromaOffsetL1[refIdxL1][cIdx - 1] << (bitDepth - 8)
```

The pixel value pbSamples[x][y] of the prediction image with x=0, . . . , nCbW-1 and y=0, . . . , nCbH-1 is derived as follows.

Then, in a case that predFlagL0 is equal to 1 and that predFlagL1 is equal to 0, then the pixel value pbSamples[x][y] of the prediction image is derived as follows.

```
if(log2Wd >= 1)
  pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
((predSamplesL0[x][y] * w0 + 2 ^ (log2Wd - 1)) >>
log2Wd) + o0)
else
  pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
predSamplesL0[x][y] * w0 + o0)
```

Otherwise, in a case that predFlagL0 is 0 and that predFlagL1 is 1, then the pixel value pbSamples[x][y] of the prediction image is derived as follows.

```
if(log2Wd >= 1)
  pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
((predSamplesL1[x][y] * w1 + 2 ^ (log2Wd - 1)) >>
log2Wd) + o1)
else
  pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
predSamplesL1[x][y] * w1 + o1)
```

Otherwise, in a case that predFlagL0 is equal to 1 and that predFlagL1 is equal to 1, then the pixel value pbSamples[x][y] of the prediction image is derived as follows.

```
pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
(predSamplesL0[x][y] * w0 + predSamplesL1[x][y] * w1 +
((o0 + o1 + 1) << log2Wd)) >> (log2Wd + 1))
```

In the method described in NPL 1, however, for example, the flag luma_weight_10_flag[i] is present that indicates whether the weight coefficient LumaWeightL0[i] and offset value luma_offset_10[refIdxL0] of luminance for each reference picture in the L0 list are present, whereas only one right shift value corresponding to the denominator of the weight coefficient is present, the only one right shift value being luma_log2_weight_denom of luminance. Thus, in a case that no weight coefficient or offset value for the

33

reference list is present, the weighted prediction processing is performed with the value of the weight coefficient set equal to 2 to the luma_log2_weight_denom-th power and with the offset value set equal to 0. Thus, in a case that no weight coefficient or offset value for the reference list is present in L0 prediction or L1 prediction, the weighted prediction processing is performed even though the normal prediction processing is originally to be performed. In a case that no weight coefficient and offset value is present for either of the L0 list and the L1 list in bi-prediction, the weighted prediction processing is performed even though the normal bi-prediction processing is originally to be performed. As described above, there is a problem in that, in some cases the weighted prediction processing is performed even in a case that the normal prediction processing is to be performed.

Thus, in the present embodiment, the function WeightedPredFlag (slice_type, cIdx, refIdxL0, refIdxL1) is defined as follows.

```

WeightedPredFlag(slice_type, cIdx, refIdxL0, refIdxL1) =
  (slice_type == P && pps_weighted_pred_flag)?
  ((cIdx == 0)? luma_weight_l0_flag[refIdxL0]:
   chroma_weight_l0_flag[refIdxL0]):
  ((slice_type == B && pps_weighted_bipred_flag)?
   ((refIdxL0 >= 0 && refIdxL1 >= 0)?
    ((cIdx == 0)? (luma_weight_l0_flag[refIdxL0] ||
     luma_weight_l1_flag[refIdxL1]):
     (chroma_weight_l0_flag[refIdxL0] ||
     chroma_weight_l1_flag[refIdxL1]))):
   ((refIdxL0 >= 0 && refIdxL1 < 0)?
    ((cIdx == 0)? luma_weight_l0_flag[refIdxL0]:
     chroma_weight_l0_flag[refIdxL0]):
   ((refIdxL0 < 0 && refIdxL1 >= 0)?
    ((cIdx == 0)? luma_weight_l1_flag[refIdxL1]:
     chroma_weight_l1_flag[refIdxL1]):0)))

```

This function WeightedPredFlag (slice_type, cIdx, refIdxL0, refIdxL1) can be used in a case that exclusive processing with the weighted prediction is performed. The syntax of FIG. 29 is an example in which bcw_idx[x0][y0] is signaled only in a case that no coefficient for the weighted prediction is present in order to exclusively operate the BCW prediction and the weighted prediction.

Besides, in a case that no weight coefficient for luminance is present in the weighted prediction processing, it is possible to set the condition that the value of the function WeightedPredFlag (B, 0, refIdxL0, refIdxL1) is equal to 0 in a case that DMVR processing and BDOF processing are applied.

In a case that no weight coefficient is present for either of the luminance and the chrominance in the weighted prediction processing, it is possible to set the condition that the values of both the function WeightedPredFlag(B, 0, refIdxL0, refIdxL1) and WeightedPredFlag(B, 1, refIdxL0, refIdxL1) are equal to 0 in a case that the DMVR processing and the BDOF processing are applied.

For the variable weightedPredFlag indicating whether to perform the weighted prediction processing, it is only necessary to set the value of the function WeightedPredFlag (slice_type, cIdx, refIdxL0, refIdxL1).

The method described in NPL 1 also has a problem in that the weighted prediction processing is performed in a case that the normal prediction processing is actually to be performed even though the value of the variable weightedPredFlag is 1 and hence the weighted prediction processing is invoked.

34

Thus, in another embodiment of the present embodiment, the weighted prediction unit 3094 derives the variable weightedPredFlag as follows.

In a case that slice_type is equal to P, the variable weightedPredFlag is set as follows:

```

weightedPredFlag = (cIdx == 0)?luma_weight_l0_flag[refIdxL0]:
  chroma_weight_l0_flag[refIdxL0].

```

Otherwise (slice_type is equal to B), the variable is set as follows.

In a case that refIdxL0 is 0 or greater and refIdxL1 is 0 or greater, then the variable weightedPredFlag is set as follows:

```

weightedPredFlag = (cIdx = 0) ?
  (luma_weight_l0_flag [refIdxL0] | luma_weight_l1_flag [refIdxL1]):
  (chroma_weight_l0_flag[refIdxL0] || chroma_weight_l1_flag
  [refIdxL1]).

```

In a case that refIdxL0 is 0 or greater and refIdxL1 is smaller than 0, then the variable weightedPredFlag is set as follows:

```

weightedPredFlag = (cIdx == 0)?luma_weight_l0_flag[refIdxL0]:
  chroma_weight_l0_flag[refIdxL0].

```

In a case that refIdxL0 is smaller than 0 and refIdxL1 is 0 or greater, then the variable weightedPredFlag is set as follows:

```

weightedPredFlag = (cIdx == 0)?luma_weight_l1_flag[refIdxL1]:
  chroma_weight_l0_flag[refIdxL1].

```

Otherwise, the variable weightedPredFlag is set as follows:

```

weightedPredFlag = 0.

```

By the derivation as described above, it is possible to specify separately for luminance and chrominance signals that the weighted prediction is not performed in a case that no weight coefficient is present, allowing the above-described problem to be solved.

As yet another embodiment of the present embodiment, a method for solving the above-described problem within the weighted prediction processing will be described.

In a case that the value of the variable weightedPredFlag is equal to 1 and that the value of the variable bcwIdx is equal to 0, then the weighted prediction processing is performed, and the prediction image pbSamples is derived as follows.

The variable shift1 is set equal to Max(2, 14-bitDepth). The variables log2Wd, o0, o1, w0, and w1 are derived as follows.

In a case that cIdx is 0, corresponding to luminance, then the following applies.

```

log2Wd = luma_log2_weight_denom + shift1
WeightL0Flag = luma_weight_l0_flag[refIdxL0]
WeightL1Flag = luma_weight_l1_flag[refIdxL1]
w0 = Luma WeightL0[refIdxL0]

```

35

-continued

```

w1 = Luma WeightL1[refIdxL1]
o0 = luma_offset_10[refIdxL0] << (bitDepth - 8)
o1 = luma_offset_11[refIdxL1] << (bitDepth - 8)

```

Otherwise (cIdx is a chrominance not equal to 0), the following applies.

```

log2Wd = ChromaLog2WeightDenom + shift1
WeightL0Flag = chroma_weight_l0_flag[refIdxL0]
WeightL1Flag = chroma_weight_l1_flag[refIdxL1]
w0 = Chroma WeightL0[refIdxL0][cIdx - 1]
w1 = Chroma WeightL1[refIdxL1][cIdx - 1]
o0 = ChromaOffsetL0[refIdxL0][cIdx - 1] << (bitDepth - 8)
o1 = ChromaOffsetL1[refIdxL1][cIdx - 1] << (bitDepth - 8)

```

The pixel value pbSamples[x][y] of the prediction image with x=0, . . . , nCbW-1 and y=0, . . . , nCbH-1 is derived as follows.

In a case that the predFlagL0 is equal to 1 and that predFlagL1 is equal to 0, then the pixel value pbSamples[x][y] of the prediction image is derived as follows.

```

if(WeightL0Flag == 0)
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
        (predSamplesL0[x][y] + offset1) >> shift1)
else if(log2Wd >= 1)
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
        ((predSamplesL0[x][y] * w0 + 2 ^ (log2Wd - 1)) >> log2Wd) + o0)
else
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1, predSamplesL0[x][y] *
        w0 + o0)

```

Otherwise, in a case that predFlagL0 is 0 and that predFlagL1 at 1, then the pixel value pbSamples[x][y] of the prediction image is derived as follows.

```

if(WeightL1Flag == 0)
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
        (predSamplesL1[x][y] + offset1) >> shift1)
else if(log2Wd >= 1)
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
        ((predSamplesL1[x][y] * w1 + 2 ^ (log2Wd - 1)) >> log2Wd) + o1)
else
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1, predSamplesL1[x][y] *
        w1 + o1)

```

Otherwise, in a case that predFlagL0 is equal to 1 and that predFlagL1 is equal to 1, then the pixel value pbSamples[x][y] of the prediction image is derived as follows.

```

if(WeightL0Flag == 0 && WeightL1Flag == 0)
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
        (predSamplesL0[x][y] + predSamplesL1[x][y] + offset2) >> shift2)
else
    pbSamples[x][y] = Clip3(0, (1 << bitDepth) - 1,
        (predSamplesL0[x][y] * w0 + predSamplesL1[x][y] * w1 +
            ((o0 + o1 + 1) << log2Wd)) >> (log2Wd + 1))

```

By the derivation as described above, it is possible to specify separately for luminance and chrominance signals that the weighted prediction is not performed in a case that no weight coefficient is present, allowing the above-described problem to be solved.

For another problem, NPL 1 introduces a new concept referred to as a Picture header, but the concept has not been addressed.

36

Thus, as another embodiment of the present embodiment, flag information for the weighted prediction is newly introduced into the Picture header. Specifically, as illustrated in FIG. 30(b), pic_weighted_pred_flag and pic_weighted_bipred_flag are introduced into picture_header_rbsp() and for the Slice header, pred_weight_table() is invoked in a case that pic_weighted_pred_flag or pic_weighted_bipred_flag is 1, as in FIG. 30(c).

The syntax as described above allows the weighted prediction processing to be switched per Picture.

In regard to another problem, NPL 1 has the following problem: in a case that slice_type is equal to B, the variable weightedPredFlag indicating whether to perform the weighted prediction processing is set equal to pps_weighted_bipred_flag && (!dmvrFlag) defined by PPS. However, one of the conditions for setting, to TRUE, dmvrFlag corresponding to a variable indicating whether to perform DMVR processing is defined only for the weighted prediction processing for the luminance signal in which both luma_weight_10_flag[refIdxL0] and luma_weight_11_flag[refIdxL1] are 0, making conditions for application of the weighted prediction processing unclear for the processing of the chrominance signal.

Thus, as another embodiment of the present embodiment, in a case that slice_type is equal to B, the variable weightedPredFlag is set equal to pps_weighted_bipred_flag && ! (dmvrFlag && cIdx==0) defined by the PPS.

Under the above conditions, even in a case that dmvrFlag is 1, the weighted prediction processing in the case of the chrominance signal appropriately operates, and the conditions for application of the weighted prediction processing are clarified.

Alternatively, both in the DMVR processing and in the BDOF processing, the weighted prediction processing may be omitted both for luminance and for chrominance. In a case that slice_type is equal to B, weightedPredFlag is set equal to pps_weighted_bipred_flag && !dmvrFlag && !bdofoAvailableFlag defined by the PPS.

Under the above-described conditions, the conditions for application of the weighted prediction processing are clarified.

BCW Prediction

A Bi-prediction with CU-level Weights (BCW) prediction is a prediction method enabling a predetermined weight coefficient to be switched at a CU level.

The following are input: two variables nCbW and nCbH specifying the width and height of the current coding block, two arrays predSamplesL0 and predSamplesL1 of (nCbW)×(nCbH), the flags predFlagL0 and predFlagL1 indicating whether a prediction list is used, the reference indexes refIdxL0 and refIdxL1, the index bcw_idx for the BCW prediction, and the variable cIdx indicating the index of each of the luminance and chrominance components. The BCW prediction processing is performed, and the pixel value of the prediction image of the array pbSamples of (nCbW)×(nCbH) is output.

In a case that sps_bcw_enabled_flag indicating whether to use the prediction at the SPS level is TRUE and that the variable weightedPredFlag is 0 and that the reference picture indicated by the two reference indexes refIdxL0 and refIdxL1 includes no weighted prediction coefficients and that the coding block size has a given value or smaller, then bcw_idx of the syntax at the CU level is explicitly signaled, and the value of bcw_idx is input to the variable bcwIdx. In a case that bcw_idx is absent, then the variable bcwIdx is set equal to 0.

37

In a case that the variable `bewIdx` is 0, the pixel value of the prediction image is derived as follows.

$$\text{pbSamples}[x][y] = \text{Clip3}(0, (1 \ll \text{bitDepth}) - 1, (\text{predSamplesL0}[x][y] + \text{predSamplesL1}[x][y] + \text{offset2}) \gg \text{shift2})$$

Otherwise (in a case that `bcwIdx` is not equal to 0), the following applies.

The variable `w1` is set equal to `bcwWLut[bcwIdx]`.
`bcwWLut[k] = {4, 5, 3, 10, -2}`.

The variable `w0` is set equal to $(8 - w1)$. The pixel value of the prediction image is derived as follows.

$$\text{pbSamples}[x][y] = \text{Clip3}(0, (1 \ll \text{bitDepth}) - 1, (w0 * \text{predSamplesL0}[x][y] + w1 * \text{predSamplesL1}[x][y] + \text{offset3}) \gg \text{shift2} + 3))$$

In a case that the BCW prediction is used in the AMVP prediction mode, the inter prediction parameter decoder **303** decodes `bcw_idx` and transmits decoded `bcw_idx` to the BCW unit **30955**. In a case that the BCW prediction is used in the merge prediction mode, the inter prediction parameter decoder **303** decodes the merge index `merge_idx`, and the merge candidate derivation unit **30361** derives `bcwIdx` of each merge candidate. Specifically, the merge candidate derivation unit **30361** uses the weight coefficient of the adjacent block used for derivation of the merge candidate as the weight coefficient of the merge candidate used for the target block. In other words, in the merge mode, the weight coefficient used in the past is inherited as a weight coefficient for the target block.

Intra Prediction Image Generation Unit **310**

In a case that `predMode` indicates an intra prediction mode, the intra prediction image generation unit **310** performs an intra prediction by using an intra prediction parameter input from the intra prediction parameter derivation unit **304** and a reference picture read out from the reference picture memory **306**.

The inverse quantization and inverse transform processing unit **311** performs inverse quantization on a quantization transform coefficient input from the parameter decoder **302** to calculate a transform coefficient.

The addition unit **312** adds the prediction image of the block input from the prediction image generation unit **308** and the prediction error input from the inverse quantization and inverse transform processing unit **311** to each other for each pixel, and generates a decoded image of the block. The addition unit **312** stores the decoded image of the block in the reference picture memory **306**, and also outputs it to the loop filter **305**.

The inverse quantization and inverse transform processing unit **311** performs inverse quantization on a quantization transform coefficient input from the parameter decoder **302** to calculate a transform coefficient.

The addition unit **312** adds the prediction image of the block input from the prediction image generation unit **308** and the prediction error input from the inverse quantization and inverse transform processing unit **311** to each other for each pixel, and generates a decoded image of the block. The addition unit **312** stores the decoded image of the block in the reference picture memory **306**, and also outputs it to the loop filter **305**.

Configuration of Video Coding Apparatus

Next, a configuration of the video coding apparatus **11** according to the present embodiment will be described. FIG.

38

22 is a block diagram illustrating a configuration of the video coding apparatus **11** according to the present embodiment. The video coding apparatus **11** includes a prediction image generation unit **101**, a subtraction unit **102**, a transform and quantization unit **103**, an inverse quantization and inverse transform processing unit **105**, an addition unit **106**, a loop filter **107**, a prediction parameter memory (a prediction parameter storage unit, a frame memory) **108**, a reference picture memory (a reference image storage unit, a frame memory) **109**, a coding parameter determination unit **110**, a parameter coder **111**, a prediction parameter derivation unit **120**, and an entropy coder **104**.

The prediction image generation unit **101** generates a prediction image for each CU. The prediction image generation unit **101** includes the inter prediction image generation unit **309** and intra prediction image generation unit **310** already described, and description of these units is omitted.

The subtraction unit **102** subtracts a pixel value of the prediction image of a block input from the prediction image generation unit **101** from a pixel value of the image *T* to generate a prediction error. The subtraction unit **102** outputs the prediction error to the transform and quantization unit **103**.

The transform and quantization unit **103** performs a frequency transform on the prediction error input from the subtraction unit **102** to calculate a transform coefficient, and derives a quantization transform coefficient by quantization. The transform and quantization unit **103** outputs the quantization transform coefficient to the parameter coder **111** and the inverse quantization and inverse transform processing unit **105**.

The inverse quantization and inverse transform processing unit **105** is the same as the inverse quantization and inverse transform processing unit **311** (FIG. 7) in the video decoding apparatus **31**, and descriptions thereof are omitted. The calculated prediction error is output to the addition unit **106**.

The parameter coder **111** includes a header coder **1110**, a CT information coder **1111**, and a CU coder **1112** (prediction mode coder). The CU coder **1112** further includes a TU coder **1114**. General operation of each module will be described below.

The header coder **1110** performs coding processing of parameters such as header information, split information, prediction information, and quantization transform coefficients.

The CT information coder **1111** codes the QT and MT (BT, TT) split information and the like.

The CU coder **1112** codes the CU information, the prediction information, the split information, and the like.

In a case that a prediction error is included in the TU, the TU coder **1114** codes the QP update information and the quantization prediction error.

The CT information coder **1111** and the CU coder **1112** supplies, to the parameter coder **111**, syntax elements such as the inter prediction parameters (`predMode`, `merge_flag`, `merge_idx`, `inter_pred_idc`, `refIdxLX`, `mvp_LX_idx`, `mvdLX`), the intra prediction parameters (`intra_luma_mpm_flag`, `intra_luma_mpm_idx`, `intra_luma_mpm_reminder`, `intra_chroma_pred_mode`), and the quantization transform coefficient.

The parameter coder **111** inputs the quantization transform coefficient and the coding parameters (split information and prediction parameters) to the entropy coder **104**. The entropy coder **104** entropy-codes the quantization transform coefficient and the coding parameters to generate a coding stream *Te* and outputs the coding stream *Te*.

The prediction parameter derivation unit **120** is a component including the inter prediction parameter coder **112** and the intra prediction parameter coder **113**, and derives an intra prediction parameter and an intra prediction parameter from the parameters input from the coding parameter determination unit **110**. The intra prediction parameter and intra prediction parameter derived are output to the parameter coder **111**.

Configuration of Inter Prediction Parameter Coder

As illustrated in FIG. 23, the inter prediction parameter coder **112** includes a parameter coding controller **1121** and an inter prediction parameter derivation unit **303**. The inter prediction parameter derivation unit **303** has a configuration common to the video decoding apparatus. The parameter coding controller **1121** includes a merge index derivation unit **11211** and a vector candidate index derivation unit **11212**.

The merge index derivation unit **11211** derives merge candidates and the like, and outputs the merge candidates and the like to the inter prediction parameter derivation unit **303**. The vector candidate index derivation unit **11212** derives prediction vector candidates and the like, and outputs the prediction vector candidates and the like to the inter prediction parameter derivation unit **303** and the parameter coder **111**.

Configuration of Intra Prediction Parameter Coder **113**

As illustrated in FIG. 24, the intra prediction parameter coder **113** includes a parameter coding controller **1131** and the intra prediction parameter derivation unit **304**. The intra prediction parameter derivation unit **304** has a configuration common to the video decoding apparatus.

The parameter coding controller **1131** derives IntraPred-Mode Y and IntraPredModeC. Furthermore, with reference to `mpmCandList[]`, `intra_luma_mpm_flag` is determined. These prediction parameters are output to the intra prediction parameter derivation unit **304** and the parameter coder **111**.

However, unlike in the video decoding apparatus, the coding parameter determination unit **110** and the prediction parameter memory **108** provide input to the inter prediction parameter derivation unit **303** and the intra prediction parameter derivation unit **304**, and output from the inter prediction parameter derivation unit **303** and the intra prediction parameter derivation unit **304** is provided to the parameter coder **111**.

The addition unit **106** adds together, for each pixel, a pixel value for the prediction block input from the prediction image generation unit **101** and a prediction error input from the inverse quantization and inverse transform processing unit **105**, generating a decoded image. The addition unit **106** stores the generated decoded image in the reference picture memory **109**.

The loop filter **107** applies a deblocking filter, an SAO, and an ALF to the decoded image generated by the addition unit **106**. Note that the loop filter **107** need not necessarily include the above-described three types of filters, and may have a configuration of only the deblocking filter, for example.

The prediction parameter memory **108** stores the prediction parameters generated by the coding parameter determination unit **110** for each target picture and CU at a predetermined position.

The reference picture memory **109** stores the decoded image generated by the loop filter **107** for each target picture and CU at a predetermined position.

The coding parameter determination unit **110** selects one set among multiple sets of coding parameters. The coding

parameters include QT, BT, or TT split information described above, a prediction parameter, or a parameter to be coded which is generated related thereto. The prediction image generation unit **101** generates the prediction image by using these coding parameters.

The coding parameter determination unit **110** calculates, for each of the multiple sets, an RD cost value indicating the magnitude of an amount of information and a coding error. The RD cost value is, for example, the sum of a code amount and the value obtained by multiplying a coefficient λ by a square error. The code amount is an amount of information of the coding stream T_e obtained by performing entropy coding on a quantization error and a coding parameter. The square error is the sum of prediction errors calculated in the subtraction unit **102**. The coefficient λ is a real number greater than a preconfigured zero. The coding parameter determination unit **110** selects a set of coding parameters of which cost value calculated is a minimum value. The coding parameter determination unit **110** outputs the determined coding parameters to the parameter coder **111** and the prediction parameter derivation unit **120**.

Note that a computer may be used to implement some of the video coding apparatus **11** and the video decoding apparatus **31** in the above-described embodiments, for example, the entropy decoder **301**, the parameter decoder **302**, the loop filter **305**, the prediction image generation unit **308**, the inverse quantization and inverse transform processing unit **311**, the addition unit **312**, the prediction parameter derivation unit **320**, the prediction image generation unit **101**, the subtraction unit **102**, the transform and quantization unit **103**, the entropy coder **104**, the inverse quantization and inverse transform processing unit **105**, the loop filter **107**, the coding parameter determination unit **110**, a parameter coder **111**, and the prediction parameter derivation unit **120**. In that case, this configuration may be realized by recording a program for realizing such control functions on a computer-readable recording medium and causing a computer system to read the program recorded on the recording medium for execution. Note that the "computer system" mentioned here refers to a computer system built into either the video coding apparatus **11** or the video decoding apparatus **31** and is assumed to include an OS and hardware components such as a peripheral apparatus. Furthermore, a "computer-readable recording medium" refers to a portable medium such as a flexible disk, a magneto-optical disk, a ROM, a CD-ROM, and the like, and a storage device such as a hard disk built into the computer system. Moreover, the "computer-readable recording medium" may include a medium that dynamically retains a program for a short period of time, such as a communication line in a case that the program is transmitted over a network such as the Internet or over a communication line such as a telephone line, and may also include a medium that retains the program for a fixed period of time, such as a volatile memory included in the computer system functioning as a server or a client in such a case. Furthermore, the above-described program may be one for realizing some of the above-described functions, and also may be one capable of realizing the above-described functions in combination with a program already recorded in a computer system.

Furthermore, a part or all of the video coding apparatus **11** and the video decoding apparatus **31** in the embodiment described above may be realized as an integrated circuit such as a Large Scale Integration (LSI). Each function block of the video coding apparatus **11** and the video decoding apparatus **31** may be individually realized as processors, or part or all may be integrated into processors. The circuit

integration technique is not limited to LSI, and the integrated circuits for the functional blocks may be realized as dedicated circuits or a multi-purpose processor. In a case that with advances in semiconductor technology, a circuit integration technology with which an LSI is replaced appears, an integrated circuit based on the technology may be used.

The embodiment of the present invention has been described in detail above referring to the drawings, but the specific configuration is not limited to the above embodiment and various amendments can be made to a design that fall within the scope that does not depart from the gist of the present invention.

Application Examples

The above-mentioned video coding apparatus **11** and the video decoding apparatus **31** can be utilized being installed to various apparatuses performing transmission, reception, recording, and regeneration of videos. Note that, the video may be a natural video imaged by camera or the like, or may be an artificial video (including CG and GUI) generated by computer or the like.

First, referring to FIG. 2, it will be described that the above-mentioned video coding apparatus **11** and the video decoding apparatus **31** can be utilized for transmission and reception of videos.

PROD_A in FIG. 2 is a block diagram illustrating a configuration of a transmitting apparatus PROD_A equipped with the video coding apparatus **11**. As illustrated in FIG. 2(a), the transmitting apparatus PROD_A includes a coder PROD_A1 which obtains coded data by coding videos, a modulation unit PROD_A2 which obtains modulation signals by modulating carrier waves with the coded data obtained by the coder PROD_A1, and a transmitter PROD_A3 which transmits the modulation signals obtained by the modulation unit PROD_A2. The above-mentioned video coding apparatus **11** is utilized as the coder PROD_A1.

The transmitting apparatus PROD_A may further include a camera PROD_A4 that images videos, a recording medium PROD_A5 that records videos, an input terminal PROD_A6 for inputting videos from the outside, and an image processing unit A7 which generates or processes images, as supply sources of videos to be input into the coder PROD_A1. Although an example configuration in which the transmitting apparatus PROD_A includes all of the constituents is illustrated in the diagram, some of the constituents may be omitted.

Note that the recording medium PROD_A5 may record videos which are not coded or may record videos coded in a coding scheme for recording different from a coding scheme for transmission. In the latter case, a decoder (not illustrated) to decode coded data read from the recording medium PROD_A5 according to the coding scheme for recording may be present between the recording medium PROD_A5 and the coder PROD_A1.

PROD_B in FIG. 2 is a block diagram illustrating a configuration of a receiving apparatus PROD_B equipped with the video decoding apparatus **31**. As illustrated in the diagram, the receiving apparatus PROD_B includes a receiver PROD_B1 that receives modulation signals, a demodulation unit PROD_B2 that obtains coded data by demodulating the modulation signals received by the receiver PROD_B1, and a decoder PROD_B3 that obtains videos by decoding the coded data obtained by the demodulation unit PROD_B2. The above-mentioned video decoding apparatus **31** is utilized as the decoder PROD_B3.

The receiving apparatus PROD_B may further include a display PROD_B4 that displays videos, a recording medium PROD_B5 for recording the videos, and an output terminal PROD_B6 for outputting the videos to the outside, as supply destinations of the videos to be output by the decoder PROD_B3. Although an example configuration that the receiving apparatus PROD_B includes all of the constituents is illustrated in the diagram, some of the constituents may be omitted.

Note that the recording medium PROD_B5 may record videos which are not coded, or may record videos which are coded in a coding scheme for recording different from a coding scheme for transmission. In the latter case, a coder (not illustrated) that codes videos acquired from the decoder PROD_B3 according to the coding scheme for recording may be present between the decoder PROD_B3 and the recording medium PROD_B5.

Note that a transmission medium for transmitting the modulation signals may be a wireless medium or may be a wired medium. In addition, a transmission mode in which the modulation signals are transmitted may be a broadcast (here, which indicates a transmission mode in which a transmission destination is not specified in advance) or may be a communication (here, which indicates a transmission mode in which a transmission destination is specified in advance). That is, the transmission of the modulation signals may be realized by any of a wireless broadcast, a wired broadcast, a wireless communication, and a wired communication.

For example, a broadcasting station (e.g., broadcasting equipment)/receiving station (e.g., television receiver) for digital terrestrial broadcasting is an example of the transmitting apparatus PROD_A/receiving apparatus PROD_B for transmitting and/or receiving the modulation signals in the wireless broadcast. In addition, a broadcasting station (e.g., broadcasting equipment)/receiving station (e.g., television receivers) for cable television broadcasting is an example of the transmitting apparatus PROD_A/receiving apparatus PROD_B for transmitting and/or receiving the modulation signals in the wired broadcast.

In addition, a server (e.g., workstation)/client (e.g., television receiver, personal computer, smartphone) for Video On Demand (VOD) services, video hosting services and the like using the Internet is an example of the transmitting apparatus PROD_A/receiving apparatus PROD_B for transmitting and/or receiving the modulation signals in communication (usually, any of a wireless medium or a wired medium is used as a transmission medium in LAN, and the wired medium is used as a transmission medium in WAN). Here, personal computers include a desktop PC, a laptop PC, and a tablet PC. In addition, smartphones also include a multifunctional mobile telephone terminal.

A client of a video hosting service has a function of coding a video imaged with a camera and uploading the video to a server, in addition to a function of decoding coded data downloaded from a server and displaying on a display. Thus, the client of the video hosting service functions as both the transmitting apparatus PROD_A and the receiving apparatus PROD_B.

Next, referring to FIG. 3, it will be described that the above-mentioned video coding apparatus **11** and the video decoding apparatus **31** can be utilized for recording and regeneration of videos.

PROD_C in FIG. 3 is a block diagram illustrating a configuration of a recording apparatus PROD_C equipped with the above-mentioned video coding apparatus **11**. As illustrated in FIG. 3(a), the recording apparatus PROD_C

includes a coder PROD_C1 that obtains coded data by coding a video, and a writing unit PROD_C2 that writes the coded data obtained by the coder PROD_C1 in a recording medium PROD_M. The above-mentioned video coding apparatus **11** is utilized as the coder PROD_C1.

Note that the recording medium PROD_M may be (1) a type of recording medium built in the recording apparatus PROD_C such as Hard Disk Drive (HDD) or Solid State Drive (SSD), may be (2) a type of recording medium connected to the recording apparatus PROD_C such as an SD memory card or a Universal Serial Bus (USB) flash memory, and may be (3) a type of recording medium loaded in a drive apparatus (not illustrated) built in the recording apparatus PROD_C such as Digital Versatile Disc (DVD: trade name) or Blu-ray Disc (BD: trade name).

In addition, the recording apparatus PROD_C may further include a camera PROD_C3 that images a video, an input terminal PROD_C4 for inputting the video from the outside, a receiver PROD_C5 for receiving the video, and an image processing unit PROD_C6 that generates or processes images, as supply sources of the video input into the coder PROD_C1. Although an example configuration that the recording apparatus PROD_C includes all of the constituents is illustrated in the diagram, some of the constituents may be omitted.

Note that the receiver PROD_C5 may receive a video which is not coded, or may receive coded data coded in a coding scheme for transmission different from the coding scheme for recording. In the latter case, a decoder for transmission (not illustrated) that decodes coded data coded in the coding scheme for transmission may be present between the receiver PROD_C5 and the coder PROD_C1.

Examples of such recording apparatus PROD_C include, for example, a DVD recorder, a BD recorder, a Hard Disk Drive (HDD) recorder, and the like (in this case, the input terminal PROD_C4 or the receiver PROD_C5 is the main supply source of videos). In addition, a camcorder (in this case, the camera PROD_C3 is the main supply source of videos), a personal computer (in this case, the receiver PROD_C5 or the image processing unit C6 is the main supply source of videos), a smartphone (in this case, the camera PROD_C3 or the receiver PROD_C5 is the main supply source of videos), or the like is an example of the recording apparatus PROD_C as well.

FIG. 3 PROD-D is a block illustrating a configuration of a reconstruction apparatus PROD_D equipped with the above-mentioned video decoding apparatus **31**. As illustrated in the diagram, the reconstruction apparatus PROD_D includes a reading unit PROD_D1 which reads coded data written in the recording medium PROD_M, and a decoder PROD_D2 which obtains a video by decoding the coded data read by the reading unit PROD_D1. The above-mentioned video decoding apparatus **31** is utilized as the decoder PROD_D2.

Note that the recording medium PROD_M may be (1) a type of recording medium built in the reconstruction apparatus PROD_D such as HDD or SSD, may be (2) a type of recording medium connected to the reconstruction apparatus PROD_D such as an SD memory card or a USB flash memory, and may be (3) a type of recording medium loaded in a drive apparatus (not illustrated) built in the reconstruction apparatus PROD_D such as a DVD or a BD.

In addition, the reconstruction apparatus PROD_D may further include a display PROD_D3 that displays a video, an output terminal PROD_D4 for outputting the video to the outside, and a transmitter PROD_D5 that transmits the video, as the supply destinations of the video to be output by

the decoder PROD_D2. Although an example configuration that the reconstruction apparatus PROD_D includes all of the constituents is illustrated in the diagram, some of the constituents may be omitted.

Note that the transmitter PROD_D5 may transmit a video which is not coded or may transmit coded data coded in the coding scheme for transmission different from a coding scheme for recording. In the latter case, a coder (not illustrated) that codes a video in the coding scheme for transmission may be present between the decoder PROD_D2 and the transmitter PROD_D5.

Examples of the reconstruction apparatus PROD_D include, for example, a DVD player, a BD player, an HDD player, and the like (in this case, the output terminal PROD_D4 to which a television receiver, and the like are connected is the main supply destination of videos). In addition, a television receiver (in this case, the display PROD_D3 is the main supply destination of videos), a digital signage (also referred to as an electronic signboard or an electronic bulletin board, and the like, and the display PROD_D3 or the transmitter PROD_D5 is the main supply destination of videos), a desktop PC (in this case, the output terminal PROD_D4 or the transmitter PROD_D5 is the main supply destination of videos), a laptop or tablet PC (in this case, the display PROD_D3 or the transmitter PROD_D5 is the main supply destination of videos), a smartphone (in this case, the display PROD_D3 or the transmitter PROD_D5 is the main supply destination of videos), or the like is an example of the reconstruction apparatus PROD_D.

Realization by Hardware and Realization by Software

Each block of the above-mentioned video decoding apparatus **31** and the video coding apparatus **11** may be realized as a hardware by a logical circuit formed on an integrated circuit (IC chip), or may be realized as a software using a Central Processing Unit (CPU).

In the latter case, each of the above-described apparatuses includes a CPU that executes a command of a program to implement each of functions, a Read Only Memory (ROM) that stores the program, a Random Access Memory (RAM) to which the program is loaded, and a storage apparatus (recording medium), such as a memory, that stores the program and various kinds of data. In addition, an objective of the embodiment of the present invention can be achieved by supplying, to each of the apparatuses, the recording medium that records, in a computer readable form, program codes of a control program (executable program, intermediate code program, source program) of each of the apparatuses that is software for realizing the above-described functions and by reading and executing, by the computer (or a CPU or an MPU), the program codes recorded in the recording medium.

As the recording medium, for example, tapes including a magnetic tape, a cassette tape and the like, discs including a magnetic disc such as a floppy (trade name) disk/a hard disk and an optical disc such as a Compact Disc Read-Only Memory (CD-ROM)/Magneto-Optical disc (MO disc)/Mini Disc (MD)/Digital Versatile Disc (DVD: trade name)/CD Recordable (CD-R)/Blu-ray Disc (trade name), cards such as an IC card (including a memory card)/an optical card, semiconductor memories such as a mask ROM/Erasable Programmable Read-Only Memory (EPROM)/Electrically Erasable and Programmable Read-Only Memory (EEPROM: trade name)/a flash ROM, logical circuits such as a Programmable logic device (PLD) and a Field Programmable Gate Array (FPGA), or the like can be used.

In addition, each of the apparatuses is configured to be connectable to a communication network, and the program

45

codes may be supplied through the communication network. The communication network is required to be capable of transmitting the program codes, but is not limited to a particular communication network. For example, the Internet, an intranet, an extranet, a Local Area Network (LAN), an Integrated Services Digital Network (ISDN), a Value-Added Network (VAN), a Community Antenna television/Cable Television (CATV) communication network, a Virtual Private Network, a telephone network, a mobile communication network, a satellite communication network, and the like are available. In addition, a transmission medium constituting this communication network is also required to be a medium which can transmit a program code, but is not limited to a particular configuration or type of transmission medium. For example, a wired transmission medium such as Institute of Electrical and Electronic Engineers (IEEE) 1394, a USB, a power line carrier, a cable TV line, a telephone line, an Asymmetric Digital Subscriber Line (ADSL) line, and a wireless transmission medium such as infrared ray of Infrared Data Association (IrDA) or a remote control, Bluetooth (trade name), IEEE 802.11 wireless communication, High Data Rate (HDR), Near Field Communication (NFC), Digital Living Network Alliance (DLNA: trade name), a cellular telephone network, a satellite channel, a terrestrial digital broadcast network are available. Note that the embodiment of the present invention can be also realized in the form of computer data signals embedded in a carrier such that the transmission of the program codes is embodied in electronic transmission.

The embodiment of the present invention is not limited to the above-described embodiment, and various modifications are possible within the scope of the claims. That is, an embodiment obtained by combining technical means modified appropriately within the scope defined by claims is included in the technical scope of the present invention as well.

INDUSTRIAL APPLICABILITY

The embodiment of the present invention can be preferably applied to a video decoding apparatus that decodes coded data in which image data is coded, and a video coding apparatus that generates coded data in which image data is coded. The embodiment of the present invention can be preferably applied to a data structure of coded data generated by the video coding apparatus and referred to by the video decoding apparatus.

CROSS-REFERENCE OF RELATED APPLICATION

This application claims the benefit of priority to JP 2019-218449 filed on Dec. 3, 2019, which is incorporated herein by reference in its entirety.

REFERENCE SIGNS LIST

31 Image decoding apparatus
301 Entropy decoder

46

302 Parameter decoder
303 Inter prediction parameter derivation unit
304 Intra prediction parameter derivation unit
305, 107 Loop filter
306, 109 Reference picture memory
307, 108 Prediction parameter memory
308, 101 Prediction image generation unit
309 Inter prediction image generation unit
310 Intra prediction image generation unit
311, 105 Inverse quantization and inverse transform processing unit
312, 106 Addition unit
320 Prediction parameter derivation unit
11 Image coding apparatus
102 Subtraction unit
103 Transform and quantization unit
104 Entropy coder
110 Coding parameter determination unit
111 Parameter coder
112 Inter prediction parameter coder
113 Intra prediction parameter coder
120 Prediction parameter derivation unit

The invention claimed is:

1. A video decoding apparatus comprising:
 - a weighted prediction circuit configured to decode a first flag indicating presence or absence of a weight coefficient for an L0 prediction of a luminance component and a second flag indicating presence or absence of a weight coefficient for an L1 prediction of the luminance component, and a third flag indicating presence or absence of a weight coefficient for an L0 prediction of a chrominance component and a fourth flag indicating presence or absence of a weight coefficient for an L1 prediction of the chrominance component;
 - a prediction parameter derivation circuit configured to (1) derive a first scale parameter indicating whether or not an L0 reference picture specified by a first reference picture index is scaled and a second scale parameter indicating whether or not an L1 reference picture specified by a second reference picture index is scaled, and (2) perform a Decoder side Motion Vector Refinement (DMVR) processing for outputting corrected motion vectors by using two reference images of a target CU; and
 - a prediction image generation circuit configured to generate an inter prediction image by using the corrected motion vectors,
- wherein in a case that a condition is satisfied, the prediction parameter derivation circuit sets a fifth flag equal to 1,
- wherein the condition includes a value of the first scale parameter being 0, a value of the second scale parameter being 0, a value of the first flag being 0, a value of the second flag being 0, a value of the third flag being 0 and a value of the fourth flag being 0,
- wherein a value of the fifth flag equal to 1 indicates that the DMVR processing is performed.

* * * * *